

# Low-Level Design (LLD)

## Smart Education Platform (Phase 1)

**Document Version:** 1.0

### Technology Stack

- Java 21
- Spring Boot 3.x
- Spring Security 6
- PostgreSQL
- Redis
- React 19
- TypeScript
- Material UI
- Docker
- Nginx
- GitHub Actions

---

## Chapter 1 – System Architecture & Project Structure

---

### 1.1 Introduction

The **Low-Level Design (LLD)** document provides the implementation-level design of the Smart Education Platform.

Unlike the SRS, which defines *what* the system should do, the LLD defines *how* it will be implemented.

This document serves as the primary technical guide for backend, frontend, DevOps, QA, and database developers.

---

## 1.2 Architecture Overview

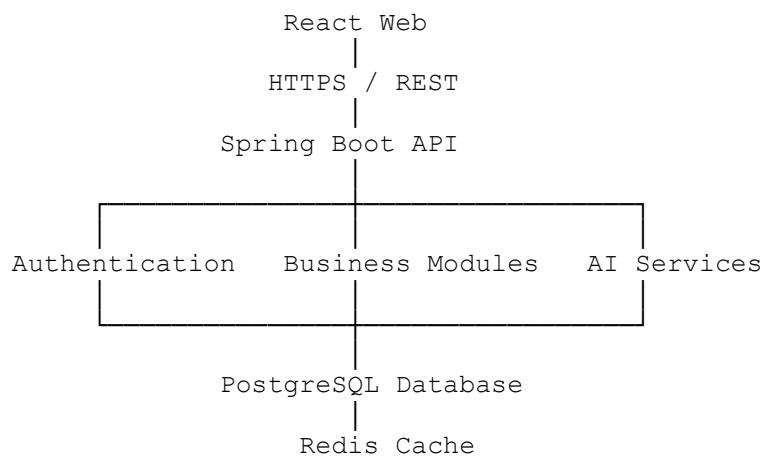
The Smart Education Platform follows a **Modular Monolithic Architecture** in Phase 1.

Every module is independent but deployed as a single Spring Boot application.

Future migration to Microservices is supported without major code changes.

---

### Architecture Diagram



---

## 1.3 High-Level Components

The application consists of the following layers.

Presentation Layer

↓

Controller Layer

↓

Service Layer

↓

Business Layer

↓

Repository Layer

↓

Database

---

## 1.4 Phase 1 Modules

### Core Modules

- Authentication
  - User Management
  - Independent Learner
  - Recruiter
  - Super Admin
- 

### Assessment Modules

- Brain Gym
  - Aptitude Arena
  - Coding Arena
  - SQL Arena
  - Debug Hero
  - Mock Interview
- 

### Career Modules

- Resume Builder
  - Placement Portal
- 

### Platform Modules

- AI Coach
  - Reports & Analytics
  - Notifications
  - Gamification
-

# 1.5 Backend Folder Structure

src/main/java

com.smarteducation

- ├── authentication
- ├── learner
- ├── recruiter
- ├── admin
- ├── braingym
- ├── aptitude
- ├── coding
- ├── sqlarena
- ├── debughero
- ├── interview
- ├── resume
- ├── placement
- ├── reports
- ├── gamification
- ├── ai
- ├── notification
- ├── common
- ├── config
- ├── security
- ├── exception
- ├── util
- └── SmartEducationApplication.java

---

# 1.6 Module Structure

Every module follows exactly the same structure.

Example

coding

- ├── controller
- ├── service
  - ├── CodingService
  - └── CodingServiceImpl
- ├── repository
- ├── entity
- ├── dto
- ├── mapper
- ├── validator
- ├── exception
- ├── specification
- └── util

This keeps every module consistent.

---

## 1.7 Layer Responsibilities

### Controller

Responsibilities

- Accept Request
- Validate Input
- Return Response
- Call Service

No business logic is allowed.

---

### Service

Responsibilities

- Business Rules
  - Transactions
  - Calculations
  - AI Integration
  - Validation
- 

### Repository

Responsibilities

- CRUD
  - Search
  - Pagination
  - Database Queries
- 

### Entity

## Responsibilities

- JPA Mapping
  - Relationships
  - Auditing
- 

# DTO

## Responsibilities

- API Request
- API Response

Entities are **never exposed** directly.

---

# Mapper

## Responsibilities

- Entity  $\rightarrow$  DTO
- DTO  $\rightarrow$  Entity

## Preferred library

- MapStruct
- 

# Validator

## Responsibilities

- Custom Validation
- Business Validation

## Example

- Password Validator
- Resume Validator
- SQL Validator

---

## 1.8 Common Package

Shared components.

common

- constants
- enums
- response
- request
- util
- mapper
- validator
- pagination
- helper

---

## 1.9 Configuration Package

config

- SwaggerConfig
- RedisConfig
- SecurityConfig
- CorsConfig
- JpaConfig
- AsyncConfig
- SchedulerConfig
- CacheConfig

---

## 1.10 Security Package

security

- JwtProvider
- JwtFilter
- JwtAuthenticationEntryPoint
- UserDetailsServiceImpl
- SecurityConfig
- PermissionEvaluator
- AuthenticationManager

---

## 1.11 Exception Package

exception

- └─ GlobalExceptionHandler
- └─ BusinessException
- └─ ResourceNotFoundException
- └─ UnauthorizedException
- └─ ValidationException
- └─ ErrorResponse
- └─ ErrorCode

---

## 1.12 Utility Package

util

- └─ DateUtil
- └─ FileUtil
- └─ JsonUtil
- └─ TokenUtil
- └─ PasswordUtil
- └─ EncryptionUtil
- └─ NumberUtil

---

## 1.13 API Layer Flow

React

↓

REST API

↓

Controller

↓

Service

↓

Repository

↓

Database

↓

Service



↓

Controller

↓

JSON Response

---

## 1.14 Dependency Injection Flow

Controller

↓

@Autowired

↓

Service

↓

@Autowired

↓

Repository

Constructor Injection is recommended instead of field injection.

---

## 1.15 Package Dependency Rules

Allowed

Controller

↓

Service

↓

Repository

Not Allowed

Repository

↓

Controller

**Not Allowed**

Controller

↓

Database

Every layer communicates only with its immediate layer.

---

## 1.16 Coding Standards

Naming Convention

Classes

CodingService

CodingController

CodingRepository

---

Methods

createChallenge()

submitChallenge()

getLeaderboard()

deleteQuestion()

---

Variables

userId

placementScore

codingLevel

---

Constants

MAX\_XP

DEFAULT\_PAGE\_SIZE

TOKEN\_EXPIRY

---

## 1.17 Design Principles

Project follows

- SOLID
  - DRY
  - KISS
  - Clean Code
  - Clean Architecture
  - Feature-Based Development
- 

## 1.18 Third-Party Integrations

Supported

- AI Provider
- Email Service
- SMS Service
- Cloud Storage
- Push Notification Service

Future integrations can be added without modifying existing modules.

---

## 1.19 Build Structure

Maven

↓

Compile

↓

Unit Test

↓

Package

↓

Docker Image

↓

Deployment

---

## 1.20 Development Guidelines

Every module must include:

- Controller
  - Service
  - Repository
  - Entity
  - DTO
  - Mapper
  - Validation
  - Exception Handling
  - Unit Tests
  - API Documentation
- 

## 1.21 Architecture Principles

- Single Responsibility Principle
  - Loose Coupling
  - High Cohesion
  - Interface-based Services
  - Stateless REST APIs
  - Immutable DTOs where practical
  - Transaction management at the service layer
- 

## 1.22 Chapter Summary

This chapter defines the foundational architecture for the Smart Education Platform. It establishes the project structure, package organization, module boundaries, coding standards, dependency rules, and development guidelines that all implementation teams will follow. This architecture ensures consistency, maintainability, scalability, and readiness for future migration from a modular monolith to microservices.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 2 – Complete Package Structure & Module-Level Design

Version: 1.0

---

### 2.1 Introduction

This chapter defines the complete project package structure, module organization, coding standards, dependency management, and feature-based architecture for the Smart Education Platform.

The project follows a **Feature-Based Modular Monolith Architecture**, where each business module is self-contained and includes its own Controller, Service, Repository, Entity, DTO, Mapper, Validation, and Exception handling.

This architecture improves:

- Maintainability
  - Scalability
  - Team Collaboration
  - Code Reusability
  - Future Microservice Migration
- 

### 2.2 Project Structure

SmartEducation

```
├── Backend (Spring Boot)
├── Frontend (React)
├── Database
├── DevOps
├── Documentation
├── Docker
└── Deployment
```

---

## 2.3 Backend Project Structure

smart-education-backend

```
├── src
├── main
│   ├── java
│   └── resources
├── test
├── pom.xml
└── Dockerfile
```

---

## 2.4 Java Package Structure

com.smarteducation

```
├── authentication
├── learner
├── recruiter
├── admin
├── gamification
├── braingym
├── aptitude
├── coding
├── sqlarena
├── debughero
├── interview
├── resume
├── placement
├── reports
├── ai
├── notification
├── dashboard
├── common
├── security
├── config
├── exception
└── scheduler
```

```
├── cache
└── util
```

---

## 2.5 Standard Module Structure

Every module follows the same structure.

### Example

coding

```
├── controller
│   ├── CodingController
│   └── LeaderboardController
├── service
│   ├── CodingService
│   └── CodingServiceImpl
├── repository
│   ├── CodingRepository
│   └── SubmissionRepository
├── entity
│   ├── CodingProblem
│   ├── CodingSubmission
│   └── CodingTestCase
├── dto
│   ├── request
│   └── response
├── mapper
├── specification
├── validator
├── enums
├── exception
└── util
```

---

## 2.6 Authentication Module

### Responsibilities

- Login

- Registration
- JWT
- Refresh Token
- Password Reset
- OTP
- Session Management

#### Package

authentication

```
|— controller
|— service
|— repository
|— entity
|— dto
|— security
|— validator
|— mapper
```

---

## 2.7 Independent Learner Module

#### Responsibilities

- Dashboard
- Profile
- Education
- Skills
- Projects
- Certifications

#### Package

learner

```
|— controller
|— service
|— repository
|— entity
|— dto
|— mapper
|— validator
```

---

## 2.8 Assessment Modules

Separate packages for



braingym

aptitude

coding

sqlarena

debughero

interview

**Every package contains**

controller

service

repository

entity

dto

mapper

validator

---

## 2.9 AI Module

ai

- |— controller
- |— service
- |— prompt
- |— provider
- |— conversation
- |— recommendation
- |— dto
- |— mapper
- |— util

**Responsibilities**

- AI Coach
- Resume Review
- Code Review
- SQL Review
- Interview Analysis
- Career Guidance

---

## 2.10 Placement Module

placement

```
|— controller
|— service
|— repository
|— entity
|— dto
|— mapper
|— validator
|— util
```

### Responsibilities

- Jobs
- Applications
- Placement Score
- Saved Jobs
- Eligibility

---

## 2.11 Recruiter Module

recruiter

```
|— controller
|— service
|— repository
|— entity
|— dto
|— mapper
|— validator
|— util
```

### Responsibilities

- Company
  - Recruiter
  - Hiring
  - Interviews
  - Offers
-

## 2.12 Super Admin Module

admin

- controller
- service
- repository
- entity
- dto
- mapper
- validator
- audit
- util

### Responsibilities

- User Management
- Question Bank
- Reports
- Configuration
- Notifications
- Platform Settings

---

## 2.13 Common Package

common

- constants
- enums
- response
- request
- mapper
- pagination
- validator
- helper
- util
- annotation

Shared by all modules.

---

## 2.14 Configuration Package

config

- SecurityConfig
- SwaggerConfig
- RedisConfig
- JpaConfig
- CacheConfig
- AsyncConfig
- SchedulerConfig
- OpenApiConfig
- CorsConfig
- JacksonConfig

---

## 2.15 Security Package

security

- JwtProvider
- JwtFilter
- JwtAuthenticationEntryPoint
- JwtAuthenticationFilter
- SecurityConfig
- CustomUserDetailsService
- RolePermissionEvaluator
- PasswordEncoderConfig

---

## 2.16 Exception Package

exception

- GlobalExceptionHandler
- BusinessException
- ValidationException
- AuthenticationException
- AuthorizationException
- ResourceNotFoundException
- ErrorCode
- ErrorResponse

---

## 2.17 Scheduler Package

scheduler

- DailyMissionScheduler
- LeaderboardScheduler
- NotificationScheduler
- ReportScheduler
- BackupScheduler

---

## 2.18 Cache Package

cache

- DashboardCache
- LeaderboardCache
- AiCache
- PlacementCache
- RedisUtility

---

## 2.19 Frontend Folder Structure

src

- api
- assets
- components
- pages
- layouts
- routes
- hooks
- services
- redux
- context
- theme
- utils
- validations
- constants
- types
- styles
- App.tsx

---

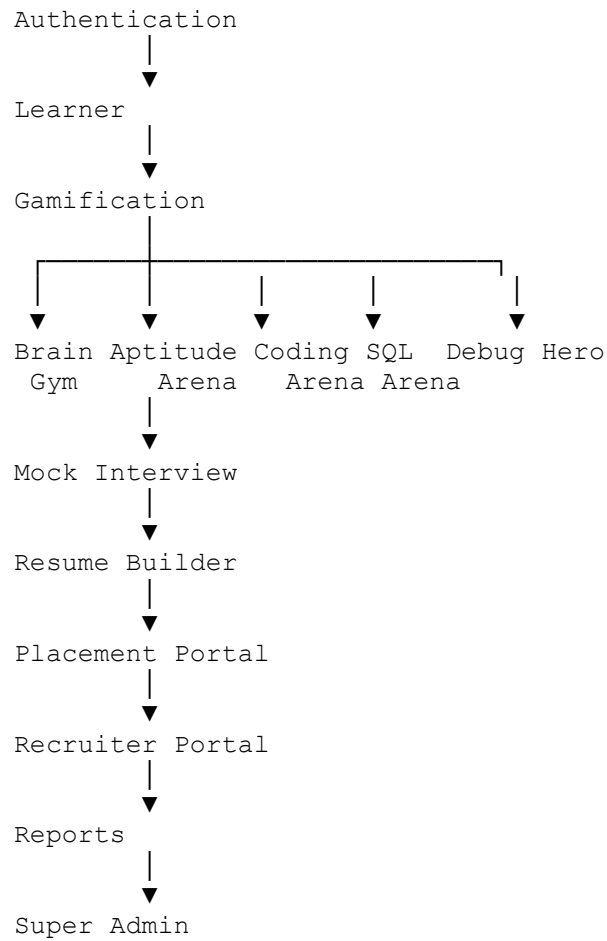
## 2.20 Component Structure

components

- common
- dashboard
- gamification
- assessment
- placement
- recruiter
- admin
- ai
- charts
- forms

---

## 2.21 Feature Module Dependency



## 2.22 Package Dependency Rules

Allowed

Controller

↓

Service

↓

Repository

↓

Database

Not Allowed

Controller

↓

Repository

Not Allowed

Repository

↓

Controller

Not Allowed

Entity

↓

Controller

---

## 2.23 Naming Standards

### Classes

CodingController

CodingService

CodingRepository

CodingSubmission

---

### Interfaces

UserService

PlacementService

InterviewService

---

### DTO

LoginRequest

RegisterRequest

CodingResponse

JobResponse

---

## Repository

UserRepository

CodingRepository

JobRepository

---

## Entity

User

CodingSubmission

JobApplication

---

## 2.24 Code Organization Standards

Each feature must contain:

- Controller
- Service Interface
- Service Implementation
- Repository
- Entity
- DTO
- Mapper
- Validation
- Exception Handling
- Unit Tests
- Integration Tests

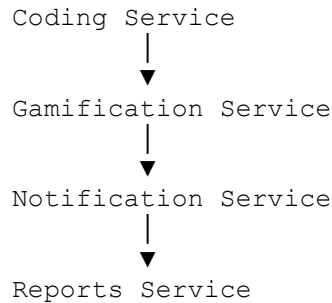
No business logic should exist in Controllers or Repositories.

---



## 2.25 Module Communication

Modules communicate only through the **Service Layer**.



This ensures loose coupling and easier maintenance.

---

## 2.26 Future Microservice Readiness

Each feature module is designed to be extracted into an independent microservice in future without major refactoring.

Planned future services include:

- Authentication Service
  - Assessment Service
  - AI Service
  - Placement Service
  - Notification Service
  - Reporting Service
  - Gamification Service
- 

## 2.27 Development Standards

Mandatory standards:

- Java 21 Coding Standards
- SOLID Principles
- Clean Architecture
- Constructor-based Dependency Injection
- Immutable DTOs where appropriate

- RESTful API Design
  - 80%+ Unit Test Coverage
  - Comprehensive API Documentation (OpenAPI)
- 

## 2.28 Chapter Summary

This chapter defines the **complete package hierarchy and module-level organization** for the Smart Education Platform. Every feature follows a consistent structure with clear responsibilities, standardized naming conventions, and controlled dependencies. The feature-based architecture promotes scalability, maintainability, team productivity, and provides a smooth path for future migration from a modular monolith to microservices.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 3 – Database Entity Design & Complete JPA Entity Architecture

**Version:** 1.0

---

### 3.1 Introduction

This chapter defines the complete JPA Entity Architecture for the Smart Education Platform. It describes how entities are modeled, related, audited, versioned, and persisted using **Spring Data JPA**, **Hibernate**, and **PostgreSQL**.

The design follows Domain-Driven Design (DDD) principles while maintaining a feature-based modular architecture.

---

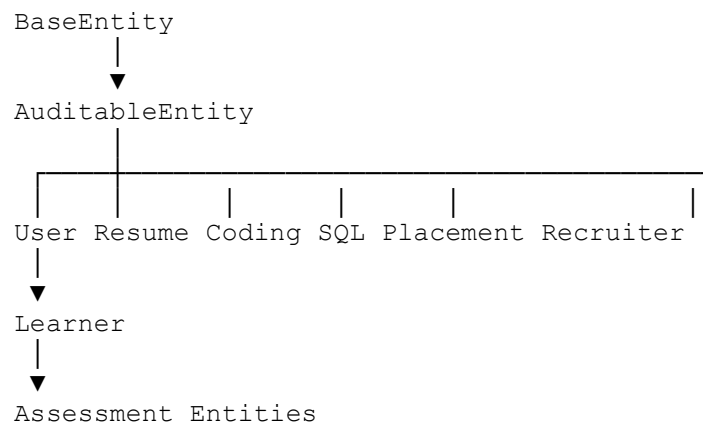
### 3.2 Objectives

The entity architecture shall:

- Use UUID as primary keys
- Support auditing
- Support optimistic locking
- Implement soft delete
- Maintain referential integrity
- Minimize database redundancy
- Support future microservices

---

## 3.3 Entity Architecture



---

## 3.4 Base Entity

Every entity inherits from **BaseEntity**.

```
@MappedSuperclass
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

}
```

### Purpose

- Common Primary Key
  - UUID Generation
  - Shared by all entities
-

## 3.5 Auditable Entity

Every business entity extends `AuditableEntity`.

```
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class AuditableEntity extends BaseEntity {

    @CreatedDate
    private LocalDateTime createdAt;

    @CreatedBy
    private UUID createdBy;

    @LastModifiedDate
    private LocalDateTime updatedAt;

    @LastModifiedBy
    private UUID updatedBy;

}
```

---

## 3.6 Soft Delete Entity

All business entities support soft deletion.

Fields

```
private Boolean deleted = false;
private LocalDateTime deletedAt;
private UUID deletedBy;
```

Deleted data remains available for auditing.

---

## 3.7 Versioning

Optimistic locking prevents concurrent updates.

```
@Version
private Long version;
```

---

## 3.8 User Entity

User

UUID id

Full Name

Email

Mobile

Password

Status

Role

CreatedAt

UpdatedAt

### Relationships

User

↓

LearnerProfile (1:1)

↓

Resume (1:N)

↓

CodingSubmission (1:N)

↓

JobApplication (1:N)

---

## 3.9 Learner Profile Entity

Stores learner information.

### Fields

- DOB
- Gender

- Address
- College
- University
- Graduation Year
- Profile Image

#### Relationship

User

↓

LearnerProfile

↓

Education

↓

Projects

↓

Certificates

---

## 3.10 Education Entity

#### Relationship

Learner

↓

Education

One learner

↓

Many education records

#### Fields

- Institute
- Degree
- Branch
- Percentage

- Start Year
- End Year

---

## 3.11 Skills Entity

Skill

↓

Java

Spring

React

SQL

Python

Relationship

User

↓

UserSkill

↓

Skill

Many-to-Many

---

## 3.12 Project Entity

Stores

- Project Name
- Description
- GitHub URL
- Technologies
- Duration

Relationship

Learner

↓

Projects

---

## 3.13 Resume Entity

Resume

↓

ResumeVersion

↓

ATS Score

One learner

↓

Multiple resumes

---

## 3.14 Assessment Entities

Separate entities for

- Brain Gym
- Aptitude
- Coding
- SQL
- Debug Hero
- Mock Interview

Each module contains

Question

↓

Attempt

↓



Answer

↓

Result

---

## 3.15 Coding Entities

CodingProblem

↓

CodingTestCase

↓

CodingSubmission

Relationships

One Problem

↓

Many Test Cases

↓

Many Submissions

---

## 3.16 SQL Entities

SqlChallenge

↓

DatabaseSchema

↓

Submission

---

## 3.17 AI Conversation Entity

## Stores

- Prompt
- Response
- Token Usage
- Response Time
- AI Model
- Conversation Type

## Relationship

User

↓

AI Conversation

---

# 3.18 Placement Entities

Company

↓

Recruiter

↓

Job

↓

Application

↓

Interview

↓

Offer

---

# 3.19 Gamification Entities

Level

Badge

XP

Coins

Mission

Reward

Leaderboard

## Relationships

User

↓

XP Transactions

↓

Badge

↓

Reward

---

## 3.20 Notification Entity

Stores

- Title
  - Message
  - Type
  - User
  - Read Status
  - Sent Date
- 

## 3.21 One-to-One Relationships

Examples

User

↓

LearnerProfile  
@OneToOne

```
@JoinColumn(name="user_id")
private User user;
```

---

## 3.22 One-to-Many Relationships

Example

User

↓

```
Resume
@OneToMany(mappedBy="user")
private List<Resume> resumes;
```

---

## 3.23 Many-to-One

Example

JobApplication

↓

```
Job
@ManyToOne
private Job job;
```

---

## 3.24 Many-to-Many

Example

User

↓

UserSkill

↓

Skill

Uses a join table.

---

## 3.25 Cascade Strategy

| Relationship         | Cascade                |
|----------------------|------------------------|
| User → Profile       | ALL                    |
| User → Resume        | ALL                    |
| Resume → Versions    | ALL                    |
| Job → Applications   | REMOVE not recommended |
| Company → Recruiters | PERSIST, MERGE         |

---

## 3.26 Fetch Strategy

Guidelines

### EAGER

- Small reference data

Examples

- Role
  - Level
- 

### LAZY

Large collections

- Resume
  - Job Applications
  - Coding Submissions
  - AI Conversations
- 

## 3.27 Entity Lifecycle

Callbacks

@PrePersist

@PreUpdate

@PostLoad

@PostPersist

@PostUpdate

@PostRemove

Uses

- Audit
- Default Values
- Logging

---

## 3.28 Hibernate Best Practices

- Avoid N+1 Query Problem
- Use EntityGraph where appropriate
- Prefer DTO projections for APIs
- Batch fetch collections
- Avoid Open Session in View
- Keep transactions short

---

## 3.29 Repository Design

Every entity has one repository.

Example

```
public interface UserRepository
extends JpaRepository<User, UUID>,
JpaSpecificationExecutor<User>{

}
```

---

## 3.30 Naming Standards

Entities

User

Resume

CodingSubmission

JobApplication

### Tables

users

resumes

coding\_submissions

job\_applications

### Columns

created\_at

updated\_at

user\_id

company\_id

---

## 3.31 Audit Support

Every entity stores

Created By

Created Date

Updated By

Updated Date

Version

Deleted Flag

---

## 3.32 Entity Dependency Diagram

User

└─ LearnerProfile

|   |                 |
|---|-----------------|
| — | Resume          |
| — | Education       |
| — | Skill           |
| — | Project         |
| — | Certification   |
| — | BrainGym        |
| — | Aptitude        |
| — | Coding          |
| — | SQL             |
| — | DebugHero       |
| — | MockInterview   |
| — | Placement       |
| — | Reports         |
| — | AI Conversation |

---

## 3.33 Entity Design Principles

The platform follows these principles:

- One Responsibility Per Entity
  - Minimal Entity Coupling
  - Bidirectional relationships only when necessary
  - UUID-based identifiers
  - Immutable reference entities where applicable
  - Domain-driven entity organization
  - Service layer manages business rules
- 

## 3.34 Acceptance Criteria

The entity architecture shall be accepted when:

- All entities inherit from `BaseEntity`.
  - Business entities inherit from `AuditableEntity`.
  - UUID primary keys are used consistently.
  - Soft delete is implemented where required.
  - Entity relationships are correctly mapped.
  - Lazy/Eager fetch strategies are appropriately applied.
  - Optimistic locking prevents concurrent update conflicts.
  - Repository interfaces follow Spring Data JPA standards.
  - Entity lifecycle callbacks function correctly.
-



## 3.35 Chapter Summary

The **Database Entity Design & Complete JPA Entity Architecture** establishes a robust persistence model for the Smart Education Platform. By combining reusable base entities, auditing, soft deletion, optimistic locking, standardized relationships, and JPA best practices, the architecture provides a scalable and maintainable foundation for all platform modules while remaining ready for future migration to a microservices architecture.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 4 – Complete DTO Design, Request/Response Models & Object Mapping Architecture

**Version:** 1.0

---

## 4.1 Introduction

The **DTO (Data Transfer Object) Architecture** defines how data is exchanged between the frontend and backend while ensuring that database entities remain isolated from external consumers.

DTOs provide:

- Secure API communication
- Clean separation between persistence and presentation layers
- Validation of client requests
- Optimized API payloads
- Consistent request and response formats

The platform follows a **Request → Service → Entity → DTO → Response** architecture.

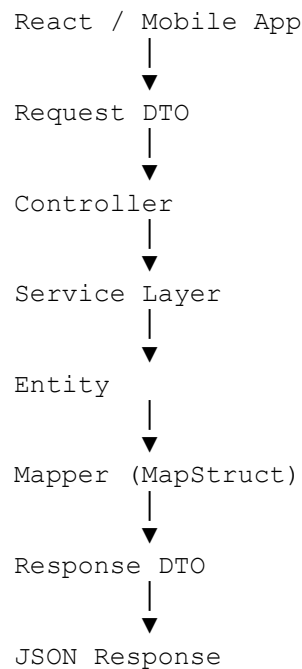
---

## 4.2 Objectives

The DTO architecture shall:

- Prevent direct exposure of JPA entities
  - Support request validation
  - Standardize API responses
  - Reduce payload size
  - Simplify frontend integration
  - Support API versioning
- 

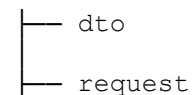
## 4.3 DTO Architecture

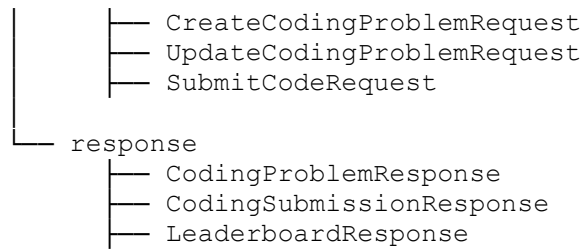


## 4.4 DTO Package Structure

Every module follows the same structure.

coding





## 4.5 DTO Categories

The platform uses the following DTO types:

### Authentication DTOs

- `LoginRequest`
  - `LoginResponse`
  - `RegisterRequest`
  - `RefreshTokenRequest`
  - `ForgotPasswordRequest`
  - `ResetPasswordRequest`
- 

### Learner DTOs

- `LearnerProfileRequest`
  - `LearnerProfileResponse`
  - `EducationRequest`
  - `SkillRequest`
  - `ProjectRequest`
  - `CertificationRequest`
- 

### Assessment DTOs

- `BrainGymRequest`
  - `AptitudeRequest`
  - `CodingSubmissionRequest`
  - `SQLSubmissionRequest`
  - `DebugSubmissionRequest`
  - `MockInterviewRequest`
-

## Placement DTOs

- JobRequest
  - JobResponse
  - ApplyJobRequest
  - JobApplicationResponse
- 

## Recruiter DTOs

- CompanyRequest
  - CompanyResponse
  - RecruiterProfileResponse
  - InterviewScheduleRequest
- 

## Admin DTOs

- UserManagementResponse
  - QuestionRequest
  - QuestionResponse
  - SystemConfigurationRequest
- 

# 4.6 Request DTO Design

## Example

```
public class LoginRequest {  
  
    @NotBlank  
    private String email;  
  
    @NotBlank  
    private String password;  
  
}
```

## Purpose

- Accept client input
- Validate data
- Pass information to the service layer

---

## 4.7 Response DTO Design

### Example

```
public class LoginResponse {  
    private UUID userId;  
    private String accessToken;  
    private String refreshToken;  
    private String role;  
}
```

### Purpose

- Return only required information
- Hide internal database fields

---

## 4.8 Generic API Response

All APIs return a common response format.

```
public class ApiResponse<T> {  
    private boolean success;  
    private String message;  
    private T data;  
    private LocalDateTime timestamp;  
}
```

---

### Example JSON

```
{  
  "success": true,  
  "message": "Login Successful",  
  "data": {  
    "accessToken": "...",
```

```
        "refreshToken": "...",  
        "role": "LEARNER"  
    },  
    "timestamp": "2026-07-10T12:00:00Z"  
}
```

---

## 4.9 Pagination Response

```
public class PageResponse<T> {  
  
    private List<T> content;  
  
    private Integer page;  
  
    private Integer size;  
  
    private Long totalElements;  
  
    private Integer totalPages;  
  
}
```

---

## 4.10 Authentication DTOs

### LoginRequest

- Email
  - Password
- 

### RegisterRequest

- Full Name
  - Email
  - Mobile
  - Password
  - Confirm Password
- 

### LoginResponse

- Access Token
- Refresh Token
- User Details

- Role
  - Expiry Time
- 

## 4.11 Learner DTOs

### Profile Response

- Personal Information
  - Education
  - Skills
  - Projects
  - Resume Status
  - Placement Score
  - Current Level
- 

## 4.12 Assessment DTOs

### CodingSubmissionRequest

- Problem ID
  - Programming Language
  - Source Code
- 

### CodingResultResponse

- Score
  - Passed Test Cases
  - Failed Test Cases
  - Execution Time
  - Memory Usage
  - Earned XP
- 

## 4.13 Placement DTOs

### JobResponse

Contains

- Job ID
  - Company Name
  - Job Title
  - Skills
  - Salary
  - Location
  - Match Percentage
- 

JobApplicationResponse

Contains

- Application ID
  - Status
  - Applied Date
  - Interview Date
- 

## 4.14 Recruiter DTOs

Recruiter Dashboard

Contains

- Active Jobs
  - Applications
  - Interviews
  - Offers
  - Hiring Analytics
- 

## 4.15 Admin DTOs

Dashboard Response

Contains

- Total Users



- Total Recruiters
  - Active Jobs
  - Active Learners
  - Platform Health
- 

## 4.16 Validation Annotations

### Standard Validation

`@NotNull`

`@NotBlank`

`@Email`

`@Size`

`@Pattern`

`@Min`

`@Max`

`@Positive`

### Custom Validation

- Password Strength
  - Mobile Number
  - File Upload
  - SQL Query Validation
- 

## 4.17 Object Mapping Architecture

### Mapping Flow

Request DTO

↓

Entity

↓

Database

↓

Entity

↓

Response DTO

Mapping Tool

**MapStruct**

Benefits

- Compile-time mapping
- High performance
- Type safety
- Less boilerplate code

---

## 4.18 Mapper Package Structure

mapper

├── UserMapper

├── ResumeMapper

├── CodingMapper

├── PlacementMapper

├── RecruiterMapper

└── AdminMapper

---

## 4.19 MapStruct Example

```
@Mapper(componentModel = "spring")
public interface UserMapper {

    User toEntity(RegisterRequest request);

    UserResponse toResponse(User user);

}
```

---

## 4.20 DTO Versioning Strategy

API versioning is maintained through URL versioning.

```
/api/v1/auth/login
```

```
/api/v2/auth/login
```

Older DTOs remain available until officially deprecated.

---

## 4.21 Error Response DTO

```
public class ErrorResponse {  
    private String errorCode;  
    private String message;  
    private List<String> validationErrors;  
    private LocalDateTime timestamp;  
}
```

---

## 4.22 File Upload DTO

Example

```
public class ResumeUploadRequest {  
    private MultipartFile resume;  
}
```

Supported Files

- PDF
- DOCX
- PNG
- JPG

---

## 4.23 DTO Naming Standards

### Request DTO

LoginRequest

RegisterRequest

CreateJobRequest

### Response DTO

LoginResponse

JobResponse

ResumeResponse

---

## 4.24 Best Practices

- Never expose JPA entities through APIs.
  - Use immutable DTOs where possible.
  - Keep DTOs lightweight.
  - Separate request and response models.
  - Validate all request DTOs.
  - Use MapStruct for mapping.
  - Avoid embedding business logic in DTOs.
  - Maintain backward compatibility for public APIs.
- 

## 4.25 Acceptance Criteria

The DTO architecture shall be accepted when:

- All API endpoints use Request and Response DTOs.
- No JPA entity is directly exposed.
- Validation annotations are applied to request DTOs.
- Generic API response format is used consistently.
- Pagination responses follow the standard model.
- MapStruct handles entity-to-DTO and DTO-to-entity conversion.
- Error responses follow a unified structure.

- DTO naming conventions are consistently applied.
- 

## 4.26 Chapter Summary

The **Complete DTO Design, Request/Response Models & Object Mapping Architecture** provides a standardized mechanism for secure and efficient data exchange across the Smart Education Platform. By separating entities from API contracts, using MapStruct for object conversion, enforcing validation, and adopting reusable response models, the architecture ensures maintainability, security, and consistency for all backend services and frontend integrations.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 5 – Repository Layer Design & Spring Data JPA Implementation

**Version:** 1.0

---

### 5.1 Introduction

The **Repository Layer** is responsible for interacting with the database and abstracts all persistence operations from the business logic. It provides a clean interface for CRUD operations, dynamic querying, pagination, sorting, projections, and optimized database access.

The Smart Education Platform uses **Spring Data JPA**, **Hibernate**, and **PostgreSQL** to implement a robust and scalable data access layer.

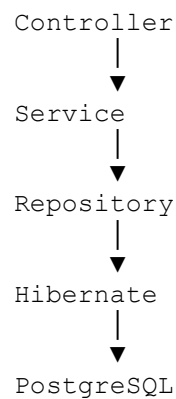
---

### 5.2 Objectives

The Repository Layer shall:

- Encapsulate all database operations
  - Minimize SQL boilerplate code
  - Support dynamic filtering
  - Enable pagination and sorting
  - Optimize query performance
  - Support transaction management
  - Be reusable across modules
- 

## 5.3 Repository Architecture

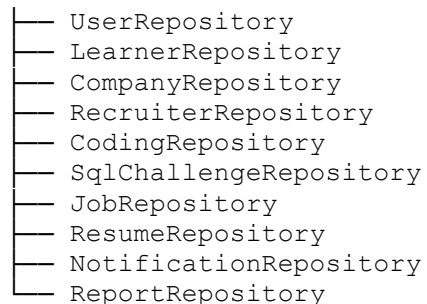


Repositories are accessed **only through the Service Layer**.

---

## 5.4 Package Structure

repository



## 5.5 Generic Repository Pattern

All repositories extend:

```
JpaRepository<Entity, UUID>
```

```
JpaSpecificationExecutor<Entity>
```

Benefits:

- CRUD operations
- Dynamic search
- Pagination
- Sorting
- Specifications

---

## 5.6 Repository Example

```
@Repository
public interface UserRepository extends
    JpaRepository<User, UUID>,
    JpaSpecificationExecutor<User> {

    Optional<User> findByEmail(String email);

    Optional<User> findByMobile(String mobile);

    boolean existsByEmail(String email);

}
```

---

## 5.7 CRUD Operations

Supported operations:

- Save
- Update
- Delete (Soft Delete)
- Find By ID
- Find All
- Exists
- Count

Example:

```
userRepository.save(user);
```

```
userRepository.findById(id);  
userRepository.existsByEmail(email);
```

---

## 5.8 Query Method Naming

Spring Data automatically generates queries.

Examples:

```
findByEmail()  
findByStatus()  
findByCompanyId()  
findByJobTitleContaining()  
findByCreatedAtBetween()  
existsByMobile()  
countByStatus()
```

---

## 5.9 Custom JPQL Queries

Example:

```
@Query("""  
SELECT u  
FROM User u  
WHERE u.status='ACTIVE'  
""")  
List<User> findActiveUsers();
```

Use JPQL for complex business queries.

---

## 5.10 Native SQL Queries

Example:

```
@Query(  

```



```
value=""  
SELECT *  
FROM users  
WHERE placement_score>80  
""",  
nativeQuery=true)  
List<User> getTopCandidates();
```

Use native SQL only when JPQL cannot provide optimal performance.

---

## 5.11 Dynamic Search using Specifications

Supports flexible filtering.

Example filters:

- Technology
- Experience
- Company
- Placement Score
- Skills
- Status

Implementation:

```
Specification<User>
```

```
Specification<Job>
```

```
Specification<CodingProblem>
```

---

## 5.12 Pagination

Standard implementation:

```
Pageable pageable =  
PageRequest.of(page, size);  
  
Page<User> users =  
userRepository.findAll(pageable);
```

Default page size:

- 20 Records

Maximum page size:

- 100 Records
- 

## 5.13 Sorting

Example:

```
Sort.by("createdAt").descending();
```

```
Sort.by("name").ascending();
```

Multiple sorting is supported.

---

## 5.14 Projection

Use projections to fetch only required fields.

Example:

```
public interface UserSummary{  
  
    String getName();  
  
    String getEmail();  
  
}
```

Benefits:

- Faster queries
  - Lower memory usage
  - Smaller API payloads
- 

## 5.15 Entity Graph

Prevent the N+1 query problem.

Example:

```
@EntityGraph(attributePaths={
    "profile",
    "skills"
})
Optional<User> findById(UUID id);
```

---

## 5.16 Transaction Management

Transactions are managed in the Service Layer.

Example:

```
@Transactional
public void applyJob() {
    ...
}
```

Repositories should not contain transaction logic.

---

## 5.17 Soft Delete Implementation

Instead of deleting records:

```
deleted=true;

deletedAt=NOW();
```

Queries automatically exclude deleted records using Hibernate filters or `@Where` clauses where appropriate.

---

## 5.18 Auditing Support

Repositories automatically persist:

- `createdAt`
- `updatedAt`
- `createdBy`
- `updatedBy`
- `version`

## 5.19 Repository Design per Module

### Authentication

- UserRepository
  - RefreshTokenRepository
  - OtpRepository
- 

### Learner

- LearnerRepository
  - EducationRepository
  - ProjectRepository
  - SkillRepository
- 

### Gamification

- BadgeRepository
  - LevelRepository
  - XPTransactionRepository
  - MissionRepository
- 

### Assessment

- BrainGymRepository
  - AptitudeRepository
  - CodingProblemRepository
  - CodingSubmissionRepository
  - SqlChallengeRepository
  - SqlSubmissionRepository
  - DebugChallengeRepository
  - MockInterviewRepository
-

## Placement

- JobRepository
  - JobApplicationRepository
  - InterviewRepository
  - OfferRepository
- 

## Recruiter

- RecruiterRepository
  - CompanyRepository
- 

## Admin

- QuestionRepository
  - ConfigurationRepository
  - AuditLogRepository
- 

# 5.20 Performance Optimization

Strategies:

- Database indexing
  - Pagination
  - Projection
  - Batch processing
  - EntityGraph
  - Redis caching
  - Avoid unnecessary joins
- 

# 5.21 Repository Naming Standards

Examples:

UserRepository

CodingProblemRepository

ResumeRepository

JobApplicationRepository

NotificationRepository

---

## 5.22 Best Practices

- Keep repositories focused on persistence only.
  - Avoid business logic in repositories.
  - Prefer derived query methods before custom queries.
  - Use JPQL before native SQL where possible.
  - Use projections for dashboard queries.
  - Implement pagination for large datasets.
  - Keep queries optimized and indexed.
  - Use Specifications for dynamic search.
- 

## 5.23 Repository Testing

Testing includes:

- CRUD Tests
- Query Method Tests
- Specification Tests
- Pagination Tests
- Sorting Tests
- Transaction Rollback Tests

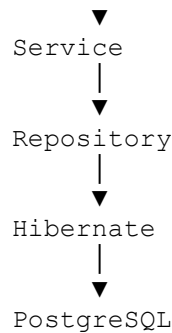
Tools:

- JUnit 5
  - Spring Boot Test
  - H2 Database (test profile)
  - Testcontainers (recommended for PostgreSQL integration tests)
- 

## 5.24 Repository Dependency Flow

Controller

|



Repositories never communicate directly with Controllers.

---

## 5.25 Acceptance Criteria

The Repository Layer shall be accepted when:

- All repositories extend `JpaRepository`.
  - Dynamic filtering is implemented using `JpaSpecificationExecutor`.
  - CRUD operations work correctly.
  - Pagination and sorting are supported.
  - JPQL and native queries are used appropriately.
  - Soft delete is implemented where required.
  - Repository methods are covered by automated tests.
  - Performance meets response time requirements.
- 

## 5.26 Chapter Summary

The **Repository Layer Design & Spring Data JPA Implementation** establishes a clean, maintainable, and high-performance data access layer for the Smart Education Platform. By leveraging Spring Data JPA, Specifications, pagination, projections, auditing, and optimized query strategies, the repository architecture provides a scalable foundation that supports all platform modules while adhering to clean architecture principles.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

# Chapter 6 – Service Layer Design, Business Logic & Transaction Management

Version: 1.0

---

## 6.1 Introduction

The **Service Layer** is the heart of the Smart Education Platform. It contains all business rules, workflows, validation logic, transaction management, and module orchestration.

The Service Layer acts as the bridge between the Controller Layer and the Repository Layer, ensuring that all business operations are centralized, reusable, testable, and maintainable.

Every business process—including learner registration, coding submissions, placement applications, gamification, AI interactions, and recruiter workflows—is implemented in this layer.

---

## 6.2 Objectives

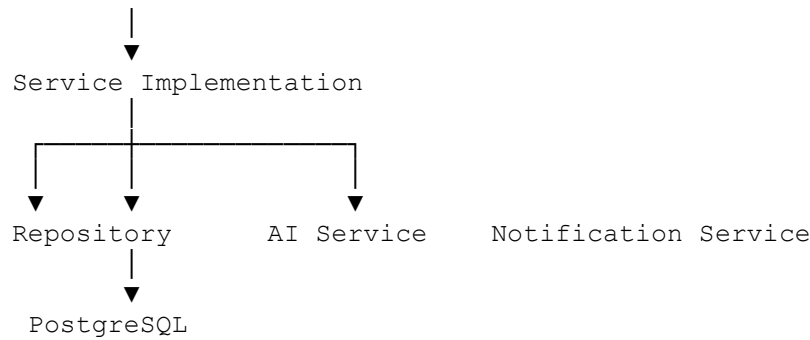
The Service Layer shall:

- Implement business logic
  - Enforce business rules
  - Manage database transactions
  - Coordinate multiple repositories
  - Integrate AI services
  - Trigger notifications
  - Update reports and analytics
  - Maintain data consistency
- 

## 6.3 Service Layer Architecture

```
REST Controller
  |
  ▼
Service Interface
```





## 6.4 Standard Service Structure

Every module follows the same structure.

coding

```
|— service
|— CodingService.java
|— CodingServiceImpl.java
```

Example

```
public interface CodingService {

    CodingSubmissionResponse submitCode(
        CodingSubmissionRequest request);

}
```

## 6.5 Service Responsibilities

The Service Layer is responsible for:

- Business logic
- Workflow execution
- Transaction handling
- Repository coordination
- Validation
- Security checks
- AI integration
- Notification triggering
- XP and reward calculation
- Audit logging

The Service Layer should never contain presentation logic.

---

## 6.6 Module-wise Services

### Authentication

- AuthenticationService
  - RegistrationService
  - OTPService
  - PasswordService
  - TokenService
- 

### Learner

- LearnerService
  - ProfileService
  - EducationService
  - SkillService
  - ProjectService
- 

### Gamification

- XPService
  - BadgeService
  - MissionService
  - RewardService
  - LeaderboardService
- 

### Assessment

- BrainGymService
  - AptitudeService
  - CodingService
  - SQLArenaService
  - DebugHeroService
  - MockInterviewService
-

## Placement

- JobService
  - JobApplicationService
  - PlacementScoreService
- 

## Recruiter

- RecruiterService
  - CompanyService
  - InterviewService
  - OfferService
- 

## Admin

- UserManagementService
  - QuestionBankService
  - ConfigurationService
  - AuditService
- 

# 6.7 Business Logic Flow

Example: Coding Submission

Submit Code

↓

Validate User

↓

Validate Challenge

↓

Compile & Execute Code

↓

Evaluate Test Cases

↓

Calculate Score

↓

Award XP

↓

Save Submission

↓

Update Leaderboard

↓

Generate Report

↓

Return Response

---

## 6.8 Service Communication

Services communicate through interfaces only.

### Example

CodingService

↓

XPService

↓

NotificationService

↓

ReportService

Avoid direct repository access between modules.

---

## 6.9 Transaction Management

All transactional operations are handled at the Service Layer.

Example

```
@Transactional
public JobApplicationResponse applyJob(
    UUID jobId) {

    ...
}
```

---

## 6.10 Transaction Types

### Read Only

```
@Transactional(readOnly = true)
```

Used for

- Dashboard
  - Reports
  - Search
  - Analytics
- 

### Read Write

```
@Transactional
```

Used for

- Registration
  - Job Application
  - Resume Creation
  - Assessment Submission
- 

## 6.11 Rollback Strategy

Transactions roll back automatically for:

- BusinessException
- RuntimeException
- DatabaseException

### Example

```
@Transactional(  
rollbackFor = Exception.class)
```

---

## 6.12 Validation Flow

Controller

↓

Request DTO Validation

↓

Business Validation

↓

Repository Validation

↓

Transaction

↓

Database

Validation includes:

- Duplicate checks
  - Eligibility
  - Authorization
  - Data integrity
- 

## 6.13 Cross-Module Integration

Example: Job Application

Placement Service

↓

Resume Service

↓

Placement Score Service

↓

Notification Service

↓

Report Service

---

## 6.14 AI Integration

Services integrate AI for:

- Resume Review
- AI Coach
- Career Guidance
- Mock Interview Analysis
- SQL Review
- Code Review

Flow

Service

↓

AI Provider

↓

Process Result

↓

Store History

↓

Return Response

---

## 6.15 Notification Integration

Business events automatically trigger notifications.

Examples:

- Registration Successful
  - Badge Earned
  - XP Awarded
  - Job Applied
  - Interview Scheduled
  - Offer Released
  - Password Changed
- 

## 6.16 Audit Logging

Every important business action is logged.

Examples

- Login
  - Registration
  - Job Posting
  - Coding Submission
  - AI Request
  - Recruiter Approval
  - Admin Configuration
- 

## 6.17 Exception Handling

Services throw custom exceptions.

Examples

`UserNotFoundException`

`JobClosedException`

`DuplicateApplicationException`

`UnauthorizedAccessException`

`BusinessException`

The Global Exception Handler converts them into standard API responses.

---



## 6.18 Service Design Principles

Each service should:

- Have one primary responsibility
  - Depend on interfaces
  - Avoid duplicate logic
  - Be stateless
  - Be independently testable
  - Avoid direct controller interaction
- 

## 6.19 Dependency Injection

Constructor Injection is mandatory.

Example

```
@Service
public class CodingServiceImpl
implements CodingService {

    private final CodingRepository repository;

    public CodingServiceImpl(
        CodingRepository repository){

        this.repository = repository;
    }
}
```

---

## 6.20 Business Rule Examples

**Job Application**

- Learner must be authenticated
  - Resume must exist
  - Job must be active
  - Duplicate applications are not allowed
  - Eligibility criteria must be satisfied
-

## Coding Arena

- Challenge must be active
  - Time limit must not expire
  - Programming language must be supported
  - Score calculated after evaluation
  - XP awarded only once per successful submission
- 

## Resume Builder

- Mandatory sections must be completed
  - ATS score recalculated after every update
  - Default resume required before job application
- 

# 6.21 Caching Strategy

Frequently accessed data:

- Dashboard
- Leaderboards
- AI Recommendations
- Question Bank
- Platform Configuration

Cache is invalidated after updates.

---

# 6.22 Asynchronous Processing

Use asynchronous execution for:

- Email sending
- Push notifications
- Report generation
- AI processing
- PDF generation

Implementation

@Async

---

## 6.23 Event-Driven Architecture (Future)

Future implementation

Coding Completed

↓

Domain Event

↓

XP Service

↓

Notification

↓

Reports

↓

Leaderboard

Using:

- Spring Events
- Kafka (Phase 2)

---

## 6.24 Service Testing

Testing includes:

- Unit Tests
- Mock Repository Tests
- Business Rule Tests
- Transaction Tests
- Exception Tests

Tools:

- JUnit 5

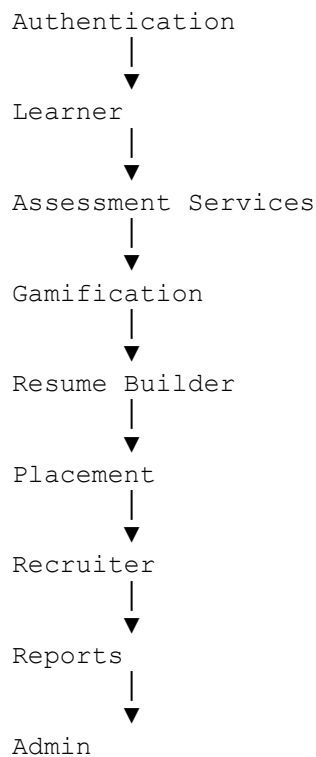
- Mockito
  - Spring Boot Test
- 

## 6.25 Performance Optimization

Strategies:

- Minimize database calls
  - Batch processing
  - Read-only transactions
  - Redis caching
  - Efficient object mapping
  - Lazy loading where appropriate
  - Asynchronous execution for non-critical tasks
- 

## 6.26 Service Dependency Diagram



## 6.27 Best Practices

- Keep Controllers thin.
  - Keep business logic inside Services.
  - Use interface-based programming.
  - Apply `@Transactional` only where needed.
  - Avoid circular service dependencies.
  - Keep methods focused on a single business operation.
  - Use custom exceptions for business errors.
  - Log important business events.
  - Write comprehensive unit tests.
- 

## 6.28 Acceptance Criteria

The Service Layer shall be accepted when:

- All business logic resides in services.
  - Controllers contain no business logic.
  - Transactions are managed correctly.
  - Cross-module workflows execute successfully.
  - AI and notification integrations function correctly.
  - Business rules are consistently enforced.
  - Service methods are covered by automated tests.
  - Performance meets defined service-level requirements.
- 

## 6.29 Chapter Summary

The **Service Layer Design, Business Logic & Transaction Management** provides the core implementation framework for the Smart Education Platform. By centralizing business rules, managing transactions, orchestrating module interactions, and integrating AI, notifications, caching, and reporting, the Service Layer ensures that all platform operations are reliable, secure, maintainable, and scalable.

## Low-Level Design (LLD)

## Smart Education Platform (Phase 1)

# Chapter 7 – Controller Layer Design, REST API Implementation & Request Processing

Version: 1.0

---

## 7.1 Introduction

The **Controller Layer** is the entry point of the Smart Education Platform backend. It receives HTTP requests from the Web and Mobile applications, validates incoming data, delegates business processing to the Service Layer, and returns standardized API responses.

The Controller Layer contains **no business logic**. It is responsible only for request processing, authentication, validation, and response formatting.

---

## 7.2 Objectives

The Controller Layer shall:

- Expose RESTful APIs
  - Accept and validate client requests
  - Delegate processing to Services
  - Return standardized responses
  - Support API versioning
  - Handle pagination and filtering
  - Secure endpoints using JWT & RBAC
- 

## 7.3 Controller Architecture

```
React / Mobile App
  |
HTTPS Request
  |
JWT Authentication Filter
  |
REST Controller
  |
Request Validation
```

Service Layer  
|  
Response DTO  
|  
JSON Response

---

## 7.4 Controller Package Structure

controller

```
├── AuthController
├── LearnerController
├── DashboardController
├── BrainGymController
├── AptitudeController
├── CodingController
├── SQLArenaController
├── DebugHeroController
├── MockInterviewController
├── ResumeController
├── PlacementController
├── RecruiterController
├── AdminController
├── AIController
├── NotificationController
└── ReportController
```

---

## 7.5 Controller Responsibilities

The Controller Layer is responsible for:

- Receiving HTTP requests
- Validating Request DTOs
- Reading JWT authentication
- Calling Service methods
- Returning Response DTOs
- Handling pagination parameters
- Returning HTTP status codes

The Controller Layer shall **not**:

- Access repositories directly
  - Implement business rules
  - Execute database queries
-

## 7.6 Request Processing Lifecycle

Client Request

↓

JWT Validation

↓

Controller

↓

DTO Validation

↓

Service

↓

Repository

↓

Database

↓

Response DTO

↓

JSON Response

---

## 7.7 Standard Controller Structure

Example

```
@RestController
@RequestMapping("/api/v1/coding")
@RequiredArgsConstructor
public class CodingController {

    private final CodingService codingService;

}
```

---



## 7.8 REST API Standards

### Base URL

/api/v1

### Example APIs

/api/v1/auth/login

/api/v1/profile

/api/v1/coding/problems

/api/v1/jobs

/api/v1/reports

---

## 7.9 HTTP Methods

| Method | Purpose              |
|--------|----------------------|
| GET    | Read Data            |
| POST   | Create Resource      |
| PUT    | Update Resource      |
| PATCH  | Partial Update       |
| DELETE | Soft Delete Resource |

---

## 7.10 Request Validation

Validation is performed using Jakarta Validation.

### Example

```
@PostMapping("/login")
public ApiResponse<LoginResponse> login(
    @Valid
    @RequestBody LoginRequest request){
```

```
...  
}
```

Supported validations:

- Required fields
- Email format
- Mobile format
- Password policy
- File size
- File type

---

## 7.11 API Response Format

Every API returns a standard structure.

```
{  
  "success": true,  
  "message": "Operation Successful",  
  "data": {},  
  "timestamp": "2026-07-10T12:30:00Z"  
}
```

---

## 7.12 HTTP Status Codes

| Status | Meaning |
|--------|---------|
|--------|---------|

|     |         |
|-----|---------|
| 200 | Success |
|-----|---------|

|     |         |
|-----|---------|
| 201 | Created |
|-----|---------|

|     |            |
|-----|------------|
| 204 | No Content |
|-----|------------|

|     |             |
|-----|-------------|
| 400 | Bad Request |
|-----|-------------|

|     |              |
|-----|--------------|
| 401 | Unauthorized |
|-----|--------------|

|     |           |
|-----|-----------|
| 403 | Forbidden |
|-----|-----------|

|     |                    |
|-----|--------------------|
| 404 | Resource Not Found |
|-----|--------------------|

| Status | Meaning               |
|--------|-----------------------|
| 409    | Conflict              |
| 422    | Validation Failed     |
| 500    | Internal Server Error |

---

## 7.13 API Versioning

Versioning Strategy

`/api/v1/...`

`/api/v2/...`

Breaking changes require a new API version.

---

## 7.14 Authentication APIs

POST `/api/v1/auth/register`

POST `/api/v1/auth/login`

POST `/api/v1/auth/logout`

POST `/api/v1/auth/refresh-token`

POST `/api/v1/auth/forgot-password`

POST `/api/v1/auth/reset-password`

---

## 7.15 Learner APIs

GET `/api/v1/profile`

PUT `/api/v1/profile`

POST `/api/v1/profile/photo`

GET `/api/v1/dashboard`

---

## 7.16 Assessment APIs

### Brain Gym

GET      /brain-gym/workouts

POST     /brain-gym/start

POST     /brain-gym/submit

### Aptitude

GET      /aptitude/tests

POST     /aptitude/start

POST     /aptitude/submit

### Coding Arena

GET      /coding/problems

GET      /coding/problem/{id}

POST     /coding/submit

GET      /coding/leaderboard

### SQL Arena

GET      /sql/challenges

POST     /sql/execute

POST     /sql/submit

---

## 7.17 Resume APIs

GET      /resume

POST     /resume

PUT      /resume

GET      /resume/download

POST     /resume/ai-review

---

## 7.18 Placement APIs

GET     /job  
GET     /job/{id}  
POST    /job/apply  
GET     /applications  
GET     /applications/{id}

---

## 7.19 Recruiter APIs

POST    /job  
PUT     /job/{id}  
DELETE  /job/{id}  
GET     /candidates  
POST    /interviews  
POST    /offers

---

## 7.20 Super Admin APIs

GET     /admin/dashboard  
GET     /admin/users  
PUT     /admin/users/{id}  
POST    /admin/questions  
PUT     /admin/configuration

---

## 7.21 Pagination & Filtering

### Example Request

GET /job?page=0&size=20&sort=createdAt,desc

### Supported Parameters

- page

- size
  - sort
  - keyword
  - status
  - technology
- 

## 7.22 File Upload APIs

Supported uploads

- Resume
- Profile Photo
- Company Logo
- Certificates

Example

```
@PostMapping(  
value="/upload",  
consumes=MediaType.MULTIPART_FORM_DATA_VALUE)
```

Maximum file size

- Resume: 5 MB
  - Image: 2 MB
- 

## 7.23 Security Annotations

Examples

```
@PreAuthorize("hasRole('LEARNER')")  
@PreAuthorize("hasRole('RECRUITER')")  
@PreAuthorize("hasRole('SUPER_ADMIN')")
```

---

## 7.24 Exception Flow

Controller

↓

Service Exception

↓

Global Exception Handler

↓

ErrorResponse

↓

JSON

---

## 7.25 OpenAPI Documentation

Every endpoint includes:

- Description
- Request Example
- Response Example
- Parameters
- Authentication Requirement
- Error Responses

Generated automatically using **OpenAPI 3 (Swagger)**.

---

## 7.26 Logging

Controller logs:

- Request received
- Authenticated user
- Response status
- Processing time
- Errors

Sensitive data such as passwords and tokens must never be logged.

---

## 7.27 Controller Testing

Testing includes:

- Endpoint Tests
- Request Validation Tests
- Authentication Tests
- Authorization Tests
- Response Format Tests
- File Upload Tests

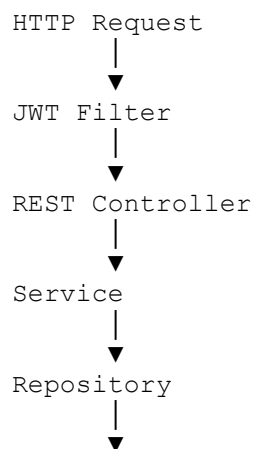
Tools:

- Spring MockMvc
  - JUnit 5
  - Mockito
- 

## 7.28 Controller Best Practices

- Keep controllers thin.
  - Use constructor injection only.
  - Validate all request DTOs.
  - Return standardized API responses.
  - Never expose entities directly.
  - Delegate all business logic to services.
  - Secure endpoints using JWT and RBAC.
  - Document all endpoints using OpenAPI.
- 

## 7.29 Controller Dependency Flow





## 7.30 Acceptance Criteria

The Controller Layer shall be accepted when:

- All REST endpoints are implemented according to standards.
  - Request validation works correctly.
  - JWT authentication and role-based authorization are enforced.
  - Standardized response and error formats are returned.
  - Pagination and filtering are supported.
  - File upload APIs validate type and size restrictions.
  - Controllers contain no business logic.
  - OpenAPI documentation is generated successfully.
  - Endpoint tests achieve the required coverage.
- 

## 7.31 Chapter Summary

The **Controller Layer Design, REST API Implementation & Request Processing** establishes a secure, standardized, and maintainable API layer for the Smart Education Platform. By separating request handling from business logic, enforcing validation, securing endpoints with JWT and RBAC, and using consistent request/response models, the Controller Layer provides a reliable interface for the Independent Learner, Recruiter, and Super Admin portals.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 8 – Spring Security, JWT Authentication, Authorization & Complete Security Architecture

Version: 1.0

---

## 8.1 Introduction

The **Security Architecture** is responsible for protecting the Smart Education Platform from unauthorized access, ensuring secure authentication, authorization, API protection, data privacy, and role-based access control.

The platform uses **Spring Security 6**, **JWT (JSON Web Token)**, **Refresh Tokens**, and **Role-Based Access Control (RBAC)** to secure all APIs.

Security is implemented across all three portals:

- Independent Learner Portal
  - Recruiter Portal
  - Super Admin Portal
- 

## 8.2 Objectives

The security architecture shall:

- Authenticate users securely
  - Authorize access based on roles
  - Protect REST APIs
  - Encrypt sensitive data
  - Prevent common web attacks
  - Support refresh tokens
  - Maintain audit logs
  - Ensure compliance with security best practices
- 

## 8.3 Security Architecture Overview

```
React / Mobile App
|
HTTPS Request
|
Security Filter Chain
|
JWT Authentication Filter
|
Authentication Manager
|
Authorization (RBAC)
|
```

```
REST Controller
  |
Service Layer
  |
Repository
  |
PostgreSQL
```

---

## 8.4 Spring Security Components

Core components include:

- SecurityFilterChain
  - AuthenticationManager
  - AuthenticationProvider
  - UserDetailsService
  - PasswordEncoder
  - JWT Authentication Filter
  - JWT Token Provider
  - AccessDeniedHandler
  - AuthenticationEntryPoint
  - SecurityContextHolder
- 

## 8.5 Authentication Flow

User Login

↓

Username & Password

↓

AuthenticationManager

↓

AuthenticationProvider

↓

UserDetailsService

↓

Password Verification

↓  
JWT Generation  
↓  
Access Token  
↓  
Refresh Token  
↓  
Return to Client

---

## 8.6 Authorization Flow

API Request  
↓  
JWT Token  
↓  
JWT Filter  
↓  
Extract User Details  
↓  
Security Context  
↓  
Role Validation  
↓  
Permission Validation  
↓  
Controller Access

---

## 8.7 JWT Token Structure

## Access Token

### Contains

- User ID
- Username
- Role
- Permissions
- Token ID
- Expiration Time

### Validity

- 30 Minutes
- 

## Refresh Token

### Contains

- User ID
- Token ID
- Expiration

### Validity

- 30 Days

Stored securely in the database.

---

# 8.8 Login Process

POST /api/v1/auth/login

↓

Validate Credentials

↓

Generate Access Token

↓

Generate Refresh Token

↓

Save Refresh Token

↓

Return Tokens

---

## 8.9 Token Refresh Process

Refresh Token

↓

Validate Token

↓

Generate New Access Token

↓

Return New Token

Old refresh tokens may be revoked during logout or suspicious activity.

---

## 8.10 Logout Process

Logout

↓

Invalidate Refresh Token

↓

Clear Security Context

↓

Logout Success

Access tokens expire naturally.

---

## 8.11 Security Filter Chain

Execution Order

HTTP Request

↓

CORS Filter

↓

JWT Authentication Filter

↓

UsernamePasswordAuthenticationFilter

↓

Authorization Filter

↓

Controller

---

## 8.12 UserDetailsService

Responsibilities

- Load user by email
  - Verify account status
  - Load roles
  - Load permissions
  - Create authenticated principal
- 

## 8.13 Password Security

Passwords are stored using:

- BCrypt Password Encoder

Rules

- Minimum 8 characters
- Uppercase letter
- Lowercase letter
- Number
- Special character

Passwords are never stored in plain text.

---

## 8.14 Role-Based Access Control (RBAC)

Supported Roles

### Independent Learner

Permissions

- View Dashboard
  - Attempt Challenges
  - Apply Jobs
  - View Reports
  - Build Resume
- 

### Recruiter

Permissions

- Manage Jobs
  - Search Candidates
  - Schedule Interviews
  - Release Offers
- 

### Super Admin

Permissions

- Manage Users
- Configure Platform
- Manage Questions
- View Audit Logs
- Manage Recruiters



---

## 8.15 Permission-Based Authorization

Permissions include:

- READ
- CREATE
- UPDATE
- DELETE
- EXPORT
- CONFIGURE

Example

```
@PreAuthorize("hasAuthority('JOB_CREATE')")
```

---

## 8.16 Method-Level Security

Examples

```
@PreAuthorize("hasRole('LEARNER')")  
@PreAuthorize("hasRole('RECRUITER')")  
@PreAuthorize("hasRole('SUPER_ADMIN')")
```

---

## 8.17 SecurityContext

After successful authentication:

JWT

↓

SecurityContextHolder

↓

Authenticated User

↓

Controller

↓

Service

Authenticated user details are available throughout the request lifecycle.

---

## 8.18 Public APIs

No authentication required

POST /auth/register

POST /auth/login

POST /auth/forgot-password

POST /auth/reset-password

GET /health

---

## 8.19 Protected APIs

Authentication required

Examples

- Dashboard
  - Coding Arena
  - SQL Arena
  - Placement Portal
  - Resume Builder
  - Recruiter Portal
  - Admin Portal
- 

## 8.20 CORS Configuration

Allowed

- Web Application
- Mobile Application
- Admin Portal

Configured

- Allowed Origins
  - Allowed Methods
  - Allowed Headers
  - Credentials Support
- 

## 8.21 CSRF Protection

For JWT-based REST APIs:

- CSRF protection is disabled because authentication is stateless.

If future server-side sessions are introduced, CSRF protection shall be enabled where applicable.

---

## 8.22 Session Management

Policy

STATELESS

Every request must include a valid JWT.

No server-side HTTP session is maintained.

---

## 8.23 Security Headers

Configured Headers

- Strict-Transport-Security (HSTS)
  - X-Content-Type-Options
  - X-Frame-Options
  - Referrer-Policy
  - Content-Security-Policy (CSP)
  - X-XSS-Protection (where supported)
-

## 8.24 API Rate Limiting

Protect against abuse.

Examples

- Login API
- OTP API
- AI Coach
- Password Reset

Future implementation

- Bucket4j
  - Redis-based Rate Limiting
- 

## 8.25 Account Protection

Security Features

- Failed login tracking
  - Temporary account lock after repeated failed attempts
  - Password reset flow
  - Email verification
  - OTP verification
  - Device tracking (future)
- 

## 8.26 Security Exception Handling

Standard Exceptions

- Invalid Credentials
- Expired Token
- Invalid Token
- Access Denied
- Account Locked
- User Disabled

Returned as standardized JSON error responses.

---

## 8.27 Audit Logging

Security Events Logged

- Login
- Logout
- Password Change
- Failed Login
- Token Refresh
- Role Change
- Permission Change
- Suspicious Activity

---

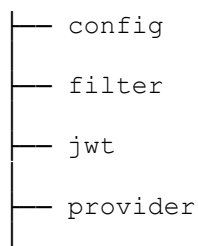
## 8.28 Security Best Practices

- HTTPS only
- BCrypt password hashing
- JWT expiration
- Refresh token rotation (recommended)
- Least privilege access
- Secure password policy
- Input validation
- SQL Injection prevention
- XSS protection
- Secure logging
- Regular dependency updates

---

## 8.29 Package Structure

security



|   |            |
|---|------------|
| — | service    |
| — | handler    |
| — | permission |
| — | annotation |
| — | util       |
| — | exception  |

---

## 8.30 Testing Strategy

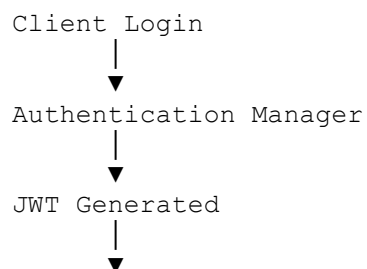
Security testing includes:

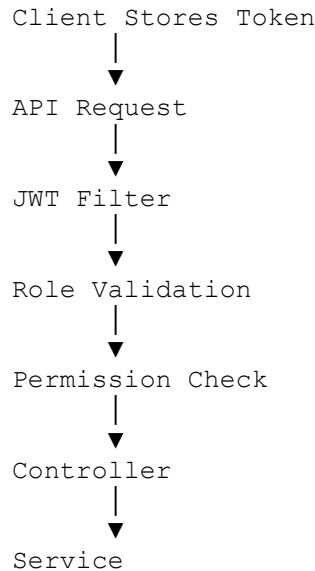
- Login success
- Invalid credentials
- Expired JWT
- Invalid JWT
- Unauthorized API access
- Role validation
- Permission validation
- Refresh token flow
- Password encryption verification
- Logout functionality

Tools

- JUnit 5
  - Spring Security Test
  - MockMvc
  - Postman
- 

## 8.31 Security Flow Diagram





## 8.32 Acceptance Criteria

The Security Architecture shall be accepted when:

- Users can authenticate using JWT.
  - Refresh tokens are issued and validated correctly.
  - Role-based and permission-based authorization are enforced.
  - Passwords are securely hashed with BCrypt.
  - Public and protected APIs are correctly configured.
  - Invalid and expired tokens are rejected.
  - Security events are logged for auditing.
  - Rate limiting protects sensitive endpoints.
  - All communication occurs over HTTPS.
  - Security tests pass successfully.
- 

## 8.33 Chapter Summary

The **Spring Security, JWT Authentication, Authorization & Complete Security Architecture** establishes a comprehensive security framework for the Smart Education Platform. By combining stateless JWT authentication, refresh tokens, RBAC, secure password handling, request filtering, audit logging, and industry-standard security controls, the platform ensures that learner, recruiter, and administrator data remains protected while providing a scalable and secure foundation for future enhancements.

# Low-Level Design (LLD)

## Smart Education Platform (Phase 1)

### Chapter 9 – Validation Framework, Global Exception Handling & Error Management

Version: 1.0

---

#### 9.1 Introduction

The **Validation Framework & Error Management** ensures that all requests entering the Smart Education Platform are validated, processed safely, and return consistent, user-friendly error responses.

The platform uses:

- Jakarta Bean Validation
- Spring Validation
- Custom Validators
- Global Exception Handler
- Standardized Error Codes
- Business Exceptions
- Audit Logging

This architecture provides consistent error handling across the Independent Learner, Recruiter, and Super Admin portals.

---

#### 9.2 Objectives

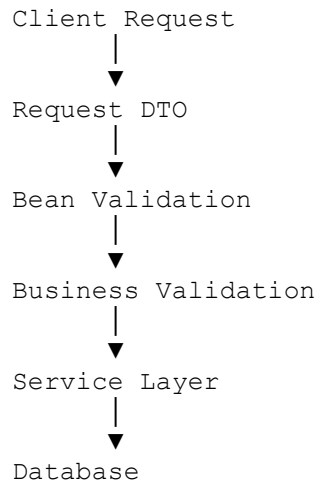
The Validation Framework shall:

- Validate all incoming requests
- Prevent invalid data from reaching the business layer
- Return standardized error responses
- Centralize exception handling



- Improve debugging and maintainability
  - Protect application integrity
- 

## 9.3 Validation Architecture



## 9.4 Validation Levels

The platform performs validation at multiple levels.

### Level 1 – Client Validation

- Mandatory fields
  - Email format
  - Password strength
  - Mobile number format
  - File size
  - File type
- 

### Level 2 – Controller Validation

Uses:

@Valid

@Validated

---

### Level 3 – Business Validation

#### Examples

- Email already exists
  - Duplicate job application
  - Challenge already completed
  - Resume required
  - Recruiter approval status
  - Job eligibility
- 

### Level 4 – Database Validation

- Primary Key
  - Foreign Key
  - Unique Constraints
  - Check Constraints
- 

## 9.5 Standard Bean Validation

Common annotations:

`@NotNull`

`@NotBlank`

`@NotEmpty`

`@Email`

`@Size`

`@Pattern`

`@Min`

`@Max`

`@Positive`

`@PositiveOrZero`

`@Past`

`@Future`

---

## 9.6 Example Validation

```
public class RegisterRequest {  
  
    @NotBlank  
    private String fullName;  
  
    @Email  
    private String email;  
  
    @Pattern(...)  
    private String mobile;  
  
    @Size(min = 8, max = 20)  
    private String password;  
  
}
```

---

## 9.7 Custom Validators

The platform defines reusable custom validators.

Examples:

- PasswordStrengthValidator
- MobileNumberValidator
- ResumeFileValidator
- SQLQueryValidator
- ProgrammingLanguageValidator
- ATSResumeValidator

Package Structure

```
validator  
├── PasswordValidator  
├── MobileValidator  
├── ResumeValidator  
├── SQLValidator  
└── CodingValidator
```

---

## 9.8 Business Validation Examples

## Registration

- Email must be unique
  - Mobile must be unique
  - Password must match policy
- 

## Job Application

- Job must be active
  - Resume must exist
  - Learner must meet eligibility criteria
  - Duplicate application not allowed
- 

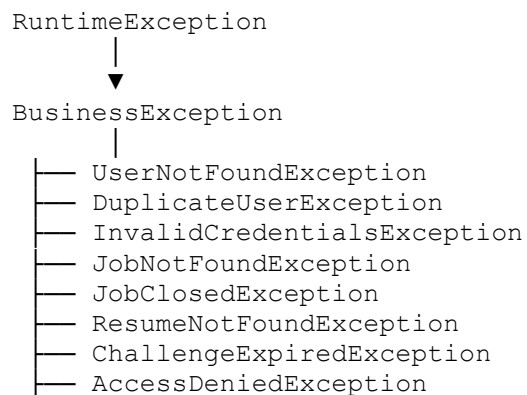
## Coding Arena

- Problem must be active
  - Language must be supported
  - Submission must be within time limit
- 

## Recruiter

- Company must be approved
  - Job title required
  - Application deadline must be valid
- 

# 9.9 Exception Hierarchy



```
└─ ValidationException
└─ AIServiceException
```

---

## 9.10 Global Exception Handler

The platform uses:

```
@RestControllerAdvice
```

Responsibilities

- Capture all exceptions
  - Convert to API responses
  - Log errors
  - Hide internal implementation details
- 

## 9.11 Standard Error Response

```
public class ErrorResponse {

    private boolean success;

    private String errorCode;

    private String message;

    private List<String> errors;

    private LocalDateTime timestamp;

}
```

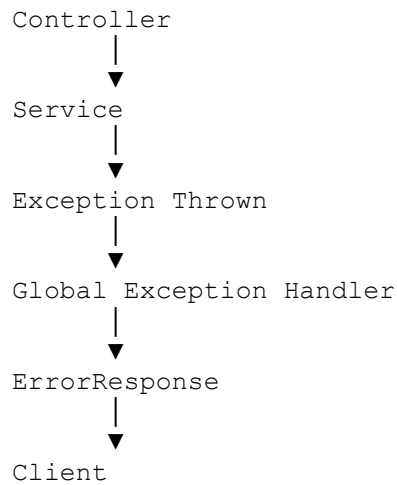
---

Example JSON

```
{
  "success": false,
  "errorCode": "JOB-004",
  "message": "Duplicate job application.",
  "errors": [
    "You have already applied for this job."
  ],
  "timestamp": "2026-07-10T14:30:00Z"
}
```

---

## 9.12 Exception Handling Flow



## 9.13 HTTP Status Mapping

| Exception               | HTTP Status |
|-------------------------|-------------|
| Validation Exception    | 400         |
| Invalid Credentials     | 401         |
| Unauthorized Access     | 403         |
| Resource Not Found      | 404         |
| Duplicate Data          | 409         |
| Business Rule Violation | 422         |
| Internal Server Error   | 500         |

## 9.14 Module-wise Error Codes

### Authentication

- AUTH-001 Invalid Credentials

- AUTH-002 Account Locked
  - AUTH-003 Invalid Token
  - AUTH-004 Token Expired
- 

## Learner

- LRN-001 Profile Not Found
  - LRN-002 Duplicate Email
  - LRN-003 Duplicate Mobile
- 

## Coding Arena

- COD-001 Problem Not Found
  - COD-002 Unsupported Language
  - COD-003 Submission Time Expired
- 

## SQL Arena

- SQL-001 Invalid Query
  - SQL-002 Execution Failed
- 

## Placement

- JOB-001 Job Not Found
  - JOB-002 Job Closed
  - JOB-003 Resume Missing
  - JOB-004 Duplicate Application
- 

## Recruiter

- REC-001 Recruiter Not Approved
  - REC-002 Company Not Found
-

## Admin

- ADM-001 Permission Denied
  - ADM-002 Configuration Error
- 

## 9.15 Validation Message Strategy

Validation messages should be:

- Clear
- User-friendly
- Actionable
- Consistent

Example

✗Invalid

Validation Failed

✓Better

Password must contain at least  
one uppercase letter,  
one lowercase letter,  
one number,  
and one special character.

---

## 9.16 Logging Strategy

The following events are logged:

- Validation failures
- Business exceptions
- Security exceptions
- Database exceptions
- Unexpected errors

Sensitive information such as passwords, JWT tokens, OTPs, and payment data must never be written to logs.

---



## 9.17 File Validation

### Supported Files

| File        | Max Size |
|-------------|----------|
| Resume      | 5 MB     |
| Image       | 2 MB     |
| Certificate | 5 MB     |

### Allowed Formats

- PDF
  - DOCX
  - JPG
  - PNG
- 

## 9.18 AI Error Handling

### Possible failures

- AI Timeout
- Service Unavailable
- Invalid Prompt
- Rate Limit Exceeded

### Fallback

AI Service

↓

Retry

↓

Fallback Message

↓

User

---

## 9.19 Database Error Handling

Handled Exceptions

- Duplicate Key
- Foreign Key Violation
- Constraint Violation
- Transaction Failure
- Optimistic Lock Failure

Converted into business-friendly responses.

---

## 9.20 Validation Sequence

Client

↓

Request DTO

↓

Bean Validation

↓

Business Validation

↓

Database Validation

↓

Success

---

## 9.21 Internationalization (Future)

Validation messages will support:

- English
- Hindi

- Additional regional languages

Messages will be stored in resource bundles for localization.

---

## 9.22 Best Practices

- Validate as early as possible.
  - Keep business validation inside Services.
  - Use custom exceptions for domain-specific errors.
  - Do not expose stack traces to clients.
  - Return consistent error codes.
  - Log exceptions with sufficient context.
  - Keep validation messages user-friendly.
  - Separate validation logic from business logic.
- 

## 9.23 Testing Strategy

Validation and exception testing includes:

- Required field validation
- Invalid format validation
- Duplicate data validation
- Authorization failures
- Business rule violations
- Global exception handler responses
- File upload validation
- API failure handling

Tools:

- JUnit 5
  - Spring Boot Test
  - MockMvc
  - Mockito
- 

## 9.24 Package Structure

exception

|   |                            |
|---|----------------------------|
| — | handler                    |
| — | BusinessException          |
| — | ValidationException        |
| — | AuthenticationException    |
| — | ResourceNotFoundException  |
| — | DuplicateResourceException |
| — | AIServiceException         |
| — | ErrorCode                  |
| — | ErrorResponse              |
| — | GlobalExceptionHandler     |

---

## 9.25 Acceptance Criteria

The Validation & Error Management framework shall be accepted when:

- All request DTOs are validated using Jakarta Bean Validation.
  - Custom validators enforce business-specific rules.
  - Global exception handling returns standardized error responses.
  - Error codes are unique and consistently applied.
  - HTTP status codes correctly represent failure conditions.
  - Sensitive information is never exposed in API responses or logs.
  - Validation and exception scenarios are covered by automated tests.
- 

## 9.26 Chapter Summary

The **Validation Framework, Global Exception Handling & Error Management** provides a centralized mechanism for validating requests, enforcing business rules, handling exceptions, and returning consistent API responses across the Smart Education Platform. By combining Bean Validation, custom validators, domain-specific exceptions, structured error codes, and centralized exception handling, the platform delivers secure, reliable, and user-friendly error management while simplifying maintenance and troubleshooting.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

# Chapter 10 – Redis Caching, Performance Optimization & Session Management

Version: 1.0

---

## 10.1 Introduction

Redis is used as the **high-speed in-memory data store** to improve application performance, reduce database load, and provide fast access to frequently requested data.

The Smart Education Platform uses Redis for:

- Dashboard Caching
- Leaderboards
- Gamification
- AI Response Cache
- OTP Storage
- Refresh Token Storage (Optional)
- Application Configuration
- Frequently Accessed Reports

Redis significantly reduces response time and improves scalability during high user traffic.

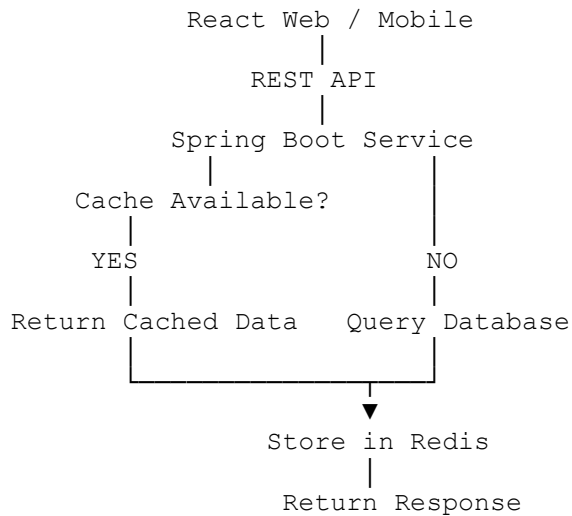
---

## 10.2 Objectives

The caching architecture shall:

- Reduce database queries
  - Improve API response time
  - Support high concurrent users
  - Cache frequently used data
  - Reduce server load
  - Improve scalability
  - Support distributed caching
-

## 10.3 Redis Architecture



## 10.4 Technology Stack

| Component          | Technology        |
|--------------------|-------------------|
| Cache Server       | Redis 7.x         |
| Spring Integration | Spring Data Redis |
| Serialization      | Jackson JSON      |
| Connection Factory | Lettuce           |
| Cache Manager      | RedisCacheManager |

## 10.5 Redis Configuration

Configuration includes:

- Redis Host
- Redis Port
- Password
- Timeout
- Database Number

- Connection Pool

Environment-specific configuration is managed through Spring profiles.

---

## 10.6 Cacheable Modules

### Authentication

- OTP
  - Refresh Token
  - Login Attempt Counter
- 

### Dashboard

- Learner Dashboard
  - Recruiter Dashboard
  - Admin Dashboard
- 

### Gamification

- Leaderboard
  - XP
  - Badges
  - Levels
- 

### Assessment

- Question Bank
  - Coding Problems
  - SQL Challenges
- 

### Placement

- Job Listings
- Company Details
- Placement Statistics

---

## AI

- Frequently Asked Questions
  - AI Recommendations
  - Resume Suggestions
- 

## 10.7 Cache Flow

API Request

↓

Service

↓

Redis

↓

Cache Found?

↓

Yes → Return Data

↓

No

↓

Database

↓

Store in Redis

↓

Return Response

---

## 10.8 Spring Cache Annotations



### Read Cache

@Cacheable

---

### Update Cache

@CachePut

---

### Remove Cache

@CacheEvict

---

### Remove All

@CacheEvict(allEntries = true)

---

## 10.9 Example

```
@Cacheable(  
value="jobs",  
key="#jobId")  
public JobResponse getJob(UUID jobId) {  
  
}
```

---

## 10.10 Cache Names

dashboard

leaderboard

jobs

coding

sql

reports

ai

resume

profile

configuration

---

## 10.11 Cache Key Convention

dashboard:user:1001

profile:user:1001

leaderboard:coding

leaderboard:aptitude

job:501

company:101

resume:user:1001

Consistent naming simplifies monitoring and invalidation.

---

## 10.12 Cache Expiration Policy

| Cache             | Expiry     |
|-------------------|------------|
| Dashboard         | 5 Minutes  |
| Leaderboard       | 10 Minutes |
| Job List          | 15 Minutes |
| AI Recommendation | 30 Minutes |
| Profile           | 30 Minutes |
| Configuration     | 12 Hours   |
| OTP               | 5 Minutes  |
| Reports           | 1 Hour     |

---

## 10.13 Cache Eviction Strategy

Cache is cleared when:

- Profile Updated
  - Resume Updated
  - Job Posted
  - Job Deleted
  - Coding Problem Updated
  - Leaderboard Reset
  - AI Recommendation Refreshed
- 

## 10.14 Performance Optimization

### Techniques

- Redis Cache
  - Pagination
  - Lazy Loading
  - Query Optimization
  - DTO Projection
  - Batch Processing
  - Connection Pooling
  - GZIP Compression
- 

## 10.15 Database Optimization

### Strategies

- Proper Indexing
  - Optimized SQL Queries
  - EntityGraph
  - Batch Inserts
  - Read-Only Transactions
  - Prepared Statements
- 

## 10.16 Session Management

The platform follows **Stateless Authentication**.

### Session Policy

STATELESS

Authentication is maintained using JWT tokens.

No HTTP Session is stored on the server.

---

## 10.17 Refresh Token Management

Refresh Tokens may be stored in Redis for fast lookup and revocation.

Flow

Login

↓

Generate Refresh Token

↓

Store in Redis

↓

Refresh API

↓

Validate Token

↓

Issue New Access Token

---

## 10.18 OTP Management

OTP data stored in Redis:

- OTP
- User ID
- Expiry
- Attempt Count

Expiry

- 5 Minutes
- 

## 10.19 Leaderboard Cache

### Cached Data

- Top Learners
- Coding Ranking
- Aptitude Ranking
- Weekly Ranking
- Monthly Ranking

### Benefits

- Fast retrieval
  - Reduced database load
- 

## 10.20 AI Cache

### Frequently cached items

- Resume Suggestions
- Interview Tips
- Career Recommendations
- Common Questions

This reduces repeated AI API calls and lowers response time.

---

## 10.21 Configuration Cache

### Frequently accessed configuration:

- Platform Settings
- Assessment Rules
- Gamification Rules
- Reward Settings

---

## 10.22 Monitoring

### Monitor

- Cache Hit Ratio
- Cache Miss Ratio
- Memory Usage
- Key Count
- Evictions
- Expired Keys
- Response Time

### Tools

- Redis Insight
- Prometheus
- Grafana

---

## 10.23 Performance Targets

| Operation          | Target      |
|--------------------|-------------|
| Dashboard Load     | < 2 Seconds |
| API Response       | < 500 ms    |
| Login              | < 2 Seconds |
| Job Search         | < 1 Second  |
| Cache Response     | < 50 ms     |
| AI Cached Response | < 200 ms    |

---

## 10.24 Security

Redis security includes:

- Password Protection
  - TLS (where supported)
  - Restricted Network Access
  - Disabled Dangerous Commands
  - Authentication Required
  - Regular Backups
- 

## 10.25 Testing Strategy

Testing includes:

- Cache Hit Validation
  - Cache Miss Validation
  - Cache Eviction
  - Cache Expiration
  - Redis Failure Recovery
  - Session Validation
  - Refresh Token Validation
  - OTP Expiry Testing
- 

## 10.26 Best Practices

- Cache only frequently accessed data.
  - Do not cache highly volatile data unless necessary.
  - Use meaningful cache names and keys.
  - Apply expiration to every cache entry.
  - Invalidate cache immediately after updates.
  - Monitor cache hit ratio regularly.
  - Keep Redis dedicated to caching and lightweight session data.
- 

## 10.27 Acceptance Criteria

The Redis Caching & Session Management module shall be accepted when:

- Frequently accessed data is cached successfully.
- Cache hit ratio meets performance goals.
- Cache invalidation works correctly after updates.
- OTP and refresh token storage function as expected.

- Dashboard and leaderboard load times improve significantly.
  - Session management remains stateless with JWT.
  - Redis monitoring and alerts are operational.
  - Performance benchmarks are achieved under expected load.
- 

## 10.28 Chapter Summary

The **Redis Caching, Performance Optimization & Session Management** architecture enhances the Smart Education Platform by providing fast, scalable, and reliable access to frequently used data. Through intelligent caching strategies, optimized database interactions, stateless JWT-based session management, and comprehensive monitoring, the platform delivers low-latency responses, supports high concurrency, and reduces infrastructure load while maintaining security and consistency.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 11 – File Storage, Document Management & Media Processing Architecture

Version: 1.0

---

### 11.1 Introduction

The **File Storage & Document Management Architecture** manages all files uploaded by learners, recruiters, and administrators while ensuring security, scalability, high availability, and efficient access.

The platform stores documents outside the application server using **MinIO (Self-Hosted)** in Phase 1, with support for **AWS S3** in future deployments.

---



## 11.2 Objectives

The File Storage System shall:

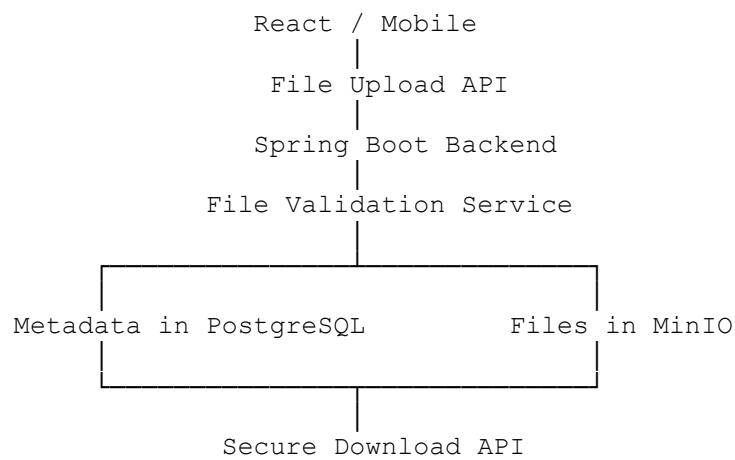
- Securely store user documents
  - Support resumés and certificates
  - Store profile images and company logos
  - Validate uploaded files
  - Prevent malicious uploads
  - Support versioning
  - Generate secure download URLs
  - Enable future cloud migration
- 

## 11.3 Supported File Types

| Category      | Formats              | Maximum Size |
|---------------|----------------------|--------------|
| Resume        | PDF, DOCX            | 5 MB         |
| Profile Photo | JPG, JPEG, PNG, WEBP | 2 MB         |
| Certificates  | PDF, JPG, PNG        | 5 MB         |
| Company Logo  | JPG, PNG, SVG        | 2 MB         |
| Documents     | PDF                  | 10 MB        |
| Reports       | PDF, XLSX            | 20 MB        |

---

## 11.4 High-Level Architecture



## 11.5 File Storage Components

The storage subsystem consists of:

- Upload Controller
  - File Service
  - Validation Service
  - Metadata Repository
  - Storage Provider (MinIO)
  - Download Service
  - Preview Service
  - Cleanup Scheduler
- 

## 11.6 Storage Folder Structure

storage



## 11.7 Database Entity

FileMetadata

Fields

- File ID
- User ID
- File Name
- Original File Name
- File Type
- MIME Type

- File Size
  - Storage Path
  - Bucket Name
  - Uploaded By
  - Upload Date
  - Status
- 

## 11.8 Upload Flow

User Selects File

↓

Frontend Validation

↓

REST API

↓

Authentication

↓

File Validation

↓

Generate Unique Name

↓

Upload to MinIO

↓

Save Metadata

↓

Return File URL

---

## 11.9 Download Flow

User Requests File

↓

Authentication

↓

Permission Validation

↓

Metadata Lookup

↓

Generate Secure URL

↓

Download File

---

## 11.10 File Naming Strategy

Internal filenames use UUIDs.

Example

9d72ab1c-9ef2-4f91-b2b2-20d8f2f4a9b7.pdf

Benefits

- Prevents filename collisions
- Improves security
- Avoids special-character issues

The original filename is stored only as metadata.

---

## 11.11 Bucket Design

Recommended Buckets

learner-files

recruiter-files

reports

public-assets

## 11.12 Validation Rules

### Validate

- File Size
- MIME Type
- Extension
- Empty File
- Corrupted File
- Duplicate Upload (Optional)

### Reject

- Executable Files
  - Script Files
  - Unsupported Formats
- 

## 11.13 Resume Management

### Each learner can:

- Upload Resume
  - Replace Resume
  - Download Resume
  - AI Review Resume
  - Track ATS Score
  - Maintain Resume Versions
- 

## 11.14 Certificate Management

### Learners can upload:

- Internship Certificates
- Training Certificates
- Achievement Certificates
- Competition Certificates

Certificates are linked to the learner profile.

---

## 11.15 Profile Image Management

Supported Features

- Upload
- Replace
- Delete
- Crop (Frontend)
- Resize
- Thumbnail Generation

Maximum Size

- 2 MB
- 

## 11.16 Company Logo Management

Recruiters can

- Upload Company Logo
- Replace Logo
- Preview Logo
- Delete Logo

Supported Formats

- PNG
  - JPG
  - SVG
- 

## 11.17 Secure File Access

Files are **not publicly accessible**.

Every download request verifies:

- User Authentication
- File Ownership
- User Role
- Access Permission

Temporary signed URLs may be generated for downloads.

---

## 11.18 Media Processing

### Image Processing

- Resize
- Compress
- Thumbnail Generation
- Metadata Extraction

### Future

- Automatic Image Optimization
  - WebP Conversion
- 

## 11.19 File Versioning

### Resume Example

Resume V1

↓

Resume V2

↓

Resume V3

↓

Current Resume

Users can restore previous versions if enabled.

---

## 11.20 Storage Lifecycle

Lifecycle

Upload

↓

Active

↓

Archive (Optional)

↓

Delete

↓

Backup

Soft delete is recommended before permanent removal.

---

## 11.21 Cleanup Scheduler

Scheduled Jobs

- Delete expired temporary files
- Remove orphan metadata
- Clean unused files
- Archive old reports

Runs daily using Spring Scheduler.

---

## 11.22 Backup Strategy

Daily Backup

- File Metadata Database

Weekly Backup



- Complete File Storage

Monthly Archive

- Reports
- Historical Documents

Backups should be stored in a separate location.

---

## 11.23 Security Measures

Security Controls

- JWT Authentication
- Role-Based Authorization
- File Type Validation
- File Size Validation
- Secure File Names
- HTTPS Only
- Signed Download URLs
- Audit Logging

Future

- Antivirus Scanning
- Malware Detection
- Data Loss Prevention (DLP)

---

## 11.24 Performance Optimization

Techniques

- CDN Support (Future)
- Image Compression
- Thumbnail Caching
- Lazy Loading
- Streaming Downloads
- Asynchronous Uploads
- Multipart Uploads for Large Files

---

## 11.25 API Design

### Upload

POST /api/v1/files/upload

### Download

GET /api/v1/files/{id}

### Delete

DELETE /api/v1/files/{id}

### Preview

GET /api/v1/files/{id}/preview

---

## 11.26 Package Structure

file

```
|— controller
|— service
|— repository
|— entity
|— dto
|— mapper
|— validator
|— storage
|— config
|— util
```

---

## 11.27 Testing Strategy

Testing includes

- Upload Success
- Invalid File Type
- Invalid File Size
- Download Authorization
- Metadata Persistence

- Storage Failure Recovery
- File Deletion
- Version Management
- Concurrent Uploads

#### Tools

- JUnit 5
  - MockMvc
  - Testcontainers (MinIO)
  - Mockito
- 

## 11.28 Best Practices

- Store files outside the application server.
  - Store metadata separately in PostgreSQL.
  - Use UUID-based filenames.
  - Never trust client-provided MIME types.
  - Validate every upload.
  - Restrict access using role-based permissions.
  - Use signed URLs for downloads.
  - Maintain audit logs for uploads and downloads.
  - Implement scheduled cleanup jobs.
  - Encrypt backups and secure storage access.
- 

## 11.29 Acceptance Criteria

The File Storage & Document Management module shall be accepted when:

- Files upload and download successfully.
  - File validation blocks unsupported or oversized files.
  - Metadata is stored correctly.
  - Secure access controls are enforced.
  - Resume versioning functions correctly.
  - Cleanup jobs remove expired temporary files.
  - Backup and recovery procedures are verified.
  - Storage performance meets platform requirements.
-

## 11.30 Chapter Summary

The **File Storage, Document Management & Media Processing Architecture** provides a secure, scalable, and maintainable solution for managing all uploaded assets within the Smart Education Platform. By separating file storage from metadata, enforcing strict validation and authorization, supporting versioning, and preparing for cloud storage and advanced media processing, the architecture ensures reliable document management for learners, recruiters, and administrators.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 12 – AI Integration Architecture, Prompt Engineering & External Service Integration

Version: 1.0

---

### 12.1 Introduction

The **AI Integration Architecture** provides intelligent assistance throughout the Smart Education Platform by integrating Large Language Models (LLMs) and AI services. The architecture is designed to be **provider-agnostic**, allowing the platform to switch between different AI providers without major code changes.

In Phase 1, AI powers:

- AI Coach
- Resume Analysis
- ATS Resume Review
- Coding Review
- SQL Query Review
- Debug Hero Assistance
- Mock Interview Evaluation
- Career Guidance
- Placement Readiness Analysis

The architecture follows an **AI Gateway Pattern**, where all AI requests pass through a unified service layer.

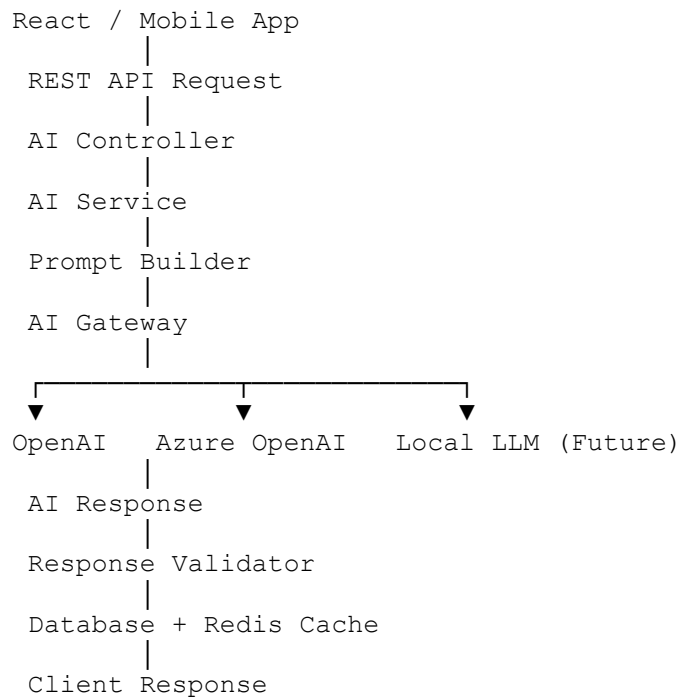
---

## 12.2 Objectives

The AI architecture shall:

- Support multiple AI providers
  - Centralize prompt management
  - Enable reusable prompt templates
  - Track token usage and cost
  - Cache AI responses where appropriate
  - Handle provider failures gracefully
  - Secure AI interactions
  - Support future AI model upgrades
- 

## 12.3 High-Level AI Architecture



## 12.4 AI Module Structure

ai

```
|— controller
|— service
|— provider
|— prompt
|— conversation
|— cache
|— repository
|— dto
|— mapper
|— validator
|— configuration
|— util
```

---

## 12.5 AI Service Components

Core services include:

- AICoachService
  - ResumeReviewService
  - ATSAAnalysisService
  - CodingReviewService
  - SQLReviewService
  - DebugAssistantService
  - MockInterviewService
  - CareerGuidanceService
  - AIUsageService
- 

## 12.6 AI Request Flow

User Question

↓

Controller

↓

AI Service

↓

Prompt Builder

↓

AI Gateway

↓

LLM Provider

↓

Response Validation

↓

Save Conversation

↓

Return Response

---

## 12.7 Prompt Engineering Architecture

Prompt generation consists of:

- System Prompt
- User Prompt
- Context Builder
- Response Format
- Output Validator

### Example

System Prompt

↓

Role Definition

↓

Context

↓

User Query

↓

Output Instructions

---

## 12.8 Prompt Template Management

Each AI feature has dedicated templates.

Examples

`resume-review.txt`

`coding-review.txt`

`sql-review.txt`

`career-guidance.txt`

`mock-interview.txt`

`ats-analysis.txt`

Templates are version-controlled and configurable.

---

## 12.9 AI Coach Integration

Features

- Career Guidance
- Technical Q&A
- Interview Preparation
- Learning Recommendations (Phase 2)
- Placement Tips

Input

- User Question
- User Profile
- Placement Score
- Skills

Output

- Personalized Guidance
- Suggested Resources
- Action Plan



---

## 12.10 Resume Analysis

AI analyzes:

- ATS Score
- Grammar
- Formatting
- Keywords
- Missing Skills
- Industry Alignment

Output

- Overall Score
- Improvement Suggestions
- Missing Keywords
- Recommended Changes

---

## 12.11 Coding Review

AI evaluates:

- Code Quality
- Logic
- Time Complexity
- Space Complexity
- Naming Conventions
- Best Practices
- Potential Bugs

The AI provides explanations and suggestions but does not replace automated code execution and test case evaluation.

---

## 12.12 SQL Review

AI analyzes:

- Query Correctness
  - Optimization Opportunities
  - JOIN Usage
  - Index Recommendations
  - Query Readability
  - SQL Best Practices
- 

## 12.13 Mock Interview Evaluation

AI evaluates:

- Technical Accuracy
- Communication Quality
- Confidence (based on available inputs)
- Problem-Solving Approach
- Answer Completeness

Outputs

- Overall Rating
  - Strengths
  - Improvement Areas
  - Suggested Practice Questions
- 

## 12.14 Debug Hero Integration

AI assists by:

- Explaining Errors
  - Suggesting Fixes
  - Identifying Common Mistakes
  - Providing Learning Tips
  - Recommending Best Practices
- 

## 12.15 Career Guidance

AI recommendations include:

- Suitable Job Roles
  - Skill Gap Analysis
  - Career Roadmap
  - Interview Readiness
  - Placement Suggestions
- 

## 12.16 Conversation Management

Each interaction stores:

- User ID
- Conversation ID
- Prompt
- AI Response
- Module
- Timestamp
- Model Used
- Token Usage

Conversation history is available to the learner unless deleted according to platform policies.

---

## 12.17 AI Gateway

The AI Gateway abstracts provider-specific APIs.

Responsibilities

- Provider Selection
  - Request Formatting
  - Authentication
  - Retry Logic
  - Timeout Handling
  - Error Handling
  - Response Standardization
- 

## 12.18 Multi-Provider Support

### Supported Providers

- OpenAI
- Azure OpenAI
- Self-hosted LLM (Future)

### Benefits

- Easy provider switching
  - Failover capability
  - Cost optimization
  - Regional deployment flexibility
- 

## 12.19 AI Response Validation

Every response is validated for:

- Empty Response
- Invalid Format
- Excessive Length
- Safety Compliance
- Response Completeness

Invalid responses trigger retry or fallback logic.

---

## 12.20 AI Response Caching

Frequently cached responses:

- Career FAQs
- Interview Tips
- Resume Suggestions
- General Java Questions
- SQL Best Practices

### Cache Duration

- 30 Minutes
-

## 12.21 Token Usage Tracking

Track

- Prompt Tokens
- Completion Tokens
- Total Tokens
- Cost Estimation
- Response Time
- Model Name

This supports monitoring and cost optimization.

---

## 12.22 External Service Integration

External integrations include:

- AI Provider APIs
- Email Service
- SMS/OTP Service
- Cloud Storage (MinIO/AWS S3)
- Push Notification Service

Each integration uses dedicated client classes and retry policies.

---

## 12.23 Retry & Fallback Strategy

AI Request

↓

Provider Failure

↓

Retry (Configurable)

↓

Secondary Provider (Future)

↓

Fallback Response

↓

User

The system returns a user-friendly message if all retries fail.

---

## 12.24 Security & Privacy

Security measures

- HTTPS communication
- Secure API keys
- Encrypted configuration
- Request authentication
- Rate limiting
- Input validation
- Audit logging

Sensitive user data should only be included in prompts when required for the requested functionality.

---

## 12.25 Package Structure

ai

```
|— controller
|— service
|— provider
|— gateway
|— prompt
|— template
|— conversation
|— cache
|— dto
|— repository
|— config
|— validator
|— util
```

---

## 12.26 Performance Optimization

### Strategies

- Redis response caching
  - Asynchronous AI calls
  - Prompt optimization
  - Streaming responses (Future)
  - Connection pooling
  - Timeout configuration
  - Batched requests where appropriate
- 

## 12.27 Monitoring

### Monitor

- Request Count
- Response Time
- Token Consumption
- Provider Availability
- Error Rate
- Retry Count
- Average Cost per Request

Dashboards can be integrated with Prometheus and Grafana.

---

## 12.28 Testing Strategy

### Testing includes:

- Prompt Template Validation
- AI Gateway Tests
- Provider Mock Tests
- Timeout Handling
- Retry Logic
- Response Validation
- Token Usage Tracking
- Cache Validation
- Integration Testing

## Tools

- JUnit 5
  - Mockito
  - WireMock (for external API mocking)
- 

## 12.29 Best Practices

- Separate prompts from source code.
  - Use reusable prompt templates.
  - Avoid sending unnecessary user data.
  - Validate AI responses before returning them.
  - Cache suitable responses.
  - Monitor token usage and costs.
  - Implement retries with exponential backoff.
  - Keep AI provider logic behind an abstraction layer.
  - Maintain prompt versioning.
- 

## 12.30 Acceptance Criteria

The AI Integration Architecture shall be accepted when:

- AI requests are routed through the AI Gateway.
  - Prompt templates are reusable and configurable.
  - Multiple AI providers can be supported.
  - AI responses are validated before delivery.
  - Conversation history is stored correctly.
  - Token usage is tracked accurately.
  - Retry and fallback mechanisms function correctly.
  - AI-related integrations are covered by automated tests.
  - Security and privacy controls are enforced.
- 

## 12.31 Chapter Summary

The **AI Integration Architecture, Prompt Engineering & External Service Integration** provides a flexible and scalable AI foundation for the Smart Education Platform. By using a provider-agnostic gateway, reusable prompt templates, structured conversation management,



response validation, caching, monitoring, and secure external integrations, the platform can deliver intelligent assistance while remaining adaptable to future AI models and evolving business requirements.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 13 – Notification System, Email, SMS, Push Notification & Event Processing Architecture

Version: 1.0

---

### 13.1 Introduction

The **Notification System** is responsible for delivering timely, secure, and reliable communications to learners, recruiters, and administrators across multiple channels.

The platform supports:

- In-App Notifications
- Email Notifications
- SMS Notifications
- OTP Verification
- Push Notifications (Future)
- Scheduled Notifications
- Event-Based Notifications

The notification architecture is designed to be **asynchronous**, scalable, and loosely coupled with the business modules.

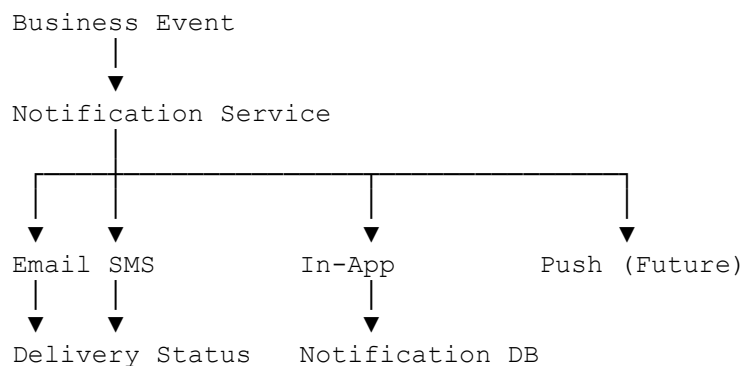
---

### 13.2 Objectives

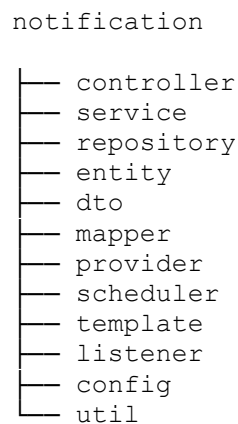
The Notification System shall:

- Deliver notifications in real time or asynchronously
  - Support multiple delivery channels
  - Provide reliable message delivery
  - Maintain notification history
  - Allow user notification preferences
  - Support retry and failure handling
  - Enable future event-driven architecture
- 

## 13.3 High-Level Notification Architecture



## 13.4 Module Structure



## 13.5 Notification Channels

Supported channels:

### In-App Notification

- Dashboard Alerts
  - XP Earned
  - Badge Earned
  - Job Updates
  - Interview Schedule
  - Offer Letter
- 

### Email

- Registration
  - Password Reset
  - Resume Review
  - Interview Invitation
  - Offer Letter
  - Account Verification
- 

### SMS

- OTP
  - Interview Reminder
  - Job Alert
  - Password Reset
- 

### Push Notification (Future)

- Daily Challenges
  - XP Updates
  - Placement Alerts
  - AI Coach Reminders
  - Mission Completion
- 

## 13.6 Notification Flow

Business Event

↓

Notification Event

↓

Notification Service

↓

Template Engine

↓

Channel Selection

↓

Email/SMS/In-App

↓

Delivery Status

↓

Notification History

---

## 13.7 Event Sources

Notifications are triggered by:

### Authentication

- Registration
  - Login Alert
  - Password Changed
  - Account Locked
  - Email Verification
- 

### Gamification

- XP Earned
- Badge Earned
- Level Up
- Reward Unlocked
- Daily Streak

---

## Assessment

- Quiz Started
- Quiz Completed
- Coding Submission
- SQL Submission
- Mock Interview Result

---

## Placement

- New Job Posted
- Job Applied
- Interview Scheduled
- Offer Released
- Application Status Changed

---

## Recruiter

- Job Published
- Candidate Applied
- Interview Accepted
- Offer Accepted

---

## Admin

- User Created
- Recruiter Approved
- Question Published
- Platform Maintenance

---

# 13.8 Notification Entity

Stores

- Notification ID
  - User ID
  - Title
  - Message
  - Channel
  - Type
  - Status
  - Priority
  - Read Status
  - Created Date
  - Delivered Date
- 

## 13.9 Notification Types

| Type    | Description          |
|---------|----------------------|
| INFO    | General Information  |
| SUCCESS | Successful Operation |
| WARNING | Warning Message      |
| ERROR   | Failure Message      |
| ALERT   | High Priority Alert  |

---

## 13.10 Notification Priority

Levels

- Critical
- High
- Medium
- Low

Critical notifications bypass batching and are delivered immediately.

---

## 13.11 Email Service

### Email Provider

- SMTP
- Microsoft 365
- Gmail Workspace
- Amazon SES (Future)

### Email Types

- Welcome Email
  - OTP
  - Password Reset
  - Resume Review
  - Interview Invite
  - Offer Letter
- 

## 13.12 SMS Service

### SMS Provider

- Twilio
- MSG91
- Fast2SMS
- AWS SNS (Future)

### Supported Messages

- OTP
  - Interview Reminder
  - Job Alert
  - Security Alert
- 

## 13.13 Push Notification Service

### Future providers

- Firebase Cloud Messaging (FCM)
- Apple Push Notification Service (APNs)

Supported

- Android
- iOS

---

## 13.14 Template Management

Templates stored separately.

Examples

registration-email.html

otp-email.html

interview-email.html

offer-email.html

job-alert-email.html

badge-earned-email.html

Supports:

- HTML Templates
- Dynamic Variables
- Localization (Future)

---

## 13.15 Template Variables

Examples

{{studentName}}

{{companyName}}

{{jobTitle}}

{{otp}}

{{interviewDate}}

{{xpEarned}}



---

## 13.16 User Notification Preferences

Users can configure:

- Email ON/OFF
- SMS ON/OFF
- In-App ON/OFF
- Push ON/OFF (Future)

Notification preferences are stored in the database and applied before delivery.

---

## 13.17 Notification Scheduling

Scheduled notifications include:

- Daily Challenge Reminder
- Interview Reminder
- Job Expiry Reminder
- Resume Update Reminder
- Streak Reminder

Implemented using Spring Scheduler.

---

## 13.18 Retry Strategy

If delivery fails:

Send

↓

Failed

↓

Retry

↓

Retry Again

↓

Mark Failed

↓

Notify Admin

Recommended retry intervals:

- 1 minute
- 5 minutes
- 15 minutes

---

## 13.19 Delivery Status

Supported statuses:

- QUEUED
- PROCESSING
- SENT
- DELIVERED
- FAILED
- READ

---

## 13.20 Notification History

Every notification stores:

- Channel
- Status
- Delivery Time
- Read Time
- Retry Count
- Failure Reason

---

## 13.21 Event Processing

Business modules publish events.

Coding Completed

↓

Notification Event

↓

Notification Service

↓

Send Notification

Future implementation:

- Spring Events
- Kafka
- RabbitMQ

---

## 13.22 Security

Notifications follow:

- JWT Authentication
- User Authorization
- Encrypted SMTP Credentials
- Secure API Keys
- OTP Expiry
- HTTPS

Sensitive information is never included unnecessarily.

---

## 13.23 Package Structure

notification

```
|— controller
|— service
|— provider
|— scheduler
|— repository
```

|  |          |
|--|----------|
|  | entity   |
|  | template |
|  | listener |
|  | event    |
|  | config   |
|  | util     |

---

## 13.24 Monitoring

Monitor:

- Email Success Rate
  - SMS Success Rate
  - Push Delivery Rate
  - Queue Size
  - Retry Count
  - Failure Rate
  - Average Delivery Time
- 

## 13.25 Performance Optimization

Strategies:

- Asynchronous processing (@Async)
  - Batch notifications
  - Template caching
  - Redis caching for preferences
  - Connection pooling
  - Queue-based processing (Future)
- 

## 13.26 Testing Strategy

Testing includes:

- Email Delivery
- SMS Delivery
- Template Rendering
- Notification Preferences
- Retry Logic
- Scheduling

- Event Processing
- Failure Handling

Tools:

- JUnit 5
  - Mockito
  - GreenMail (Email Testing)
  - Mock SMS Provider
- 

## 13.27 Best Practices

- Use asynchronous notification processing.
  - Keep notification templates externalized.
  - Avoid hardcoding messages.
  - Respect user notification preferences.
  - Retry transient failures only.
  - Log all notification events.
  - Monitor delivery metrics.
  - Separate notification logic from business logic.
- 

## 13.28 Acceptance Criteria

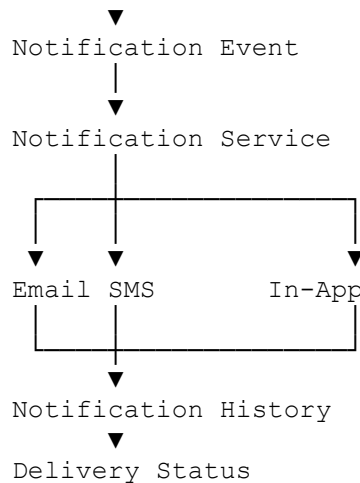
The Notification System shall be accepted when:

- In-app, email, and SMS notifications are delivered successfully.
  - Notification templates support dynamic variables.
  - User notification preferences are enforced.
  - Delivery status and history are recorded.
  - Retry mechanisms handle temporary failures.
  - Scheduled notifications execute correctly.
  - Notification events are processed asynchronously.
  - Monitoring dashboards display delivery metrics.
- 

## 13.29 Notification Processing Diagram

Business Module

|



## 13.30 Chapter Summary

The **Notification System, Email, SMS, Push Notification & Event Processing Architecture** provides a centralized and extensible communication framework for the Smart Education Platform. By supporting multiple notification channels, configurable templates, user preferences, asynchronous processing, retry mechanisms, and comprehensive monitoring, the platform ensures reliable and timely communication with learners, recruiters, and administrators while remaining ready for future event-driven and real-time enhancements.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 14 – Scheduler, Background Jobs, Batch Processing & System Automation

Version: 1.0

---

### 14.1 Introduction

The **Scheduler & Background Processing Architecture** enables the Smart Education Platform to execute automated tasks without user interaction. It is responsible for recurring jobs,

maintenance activities, report generation, cache refresh, notifications, leaderboard calculations, and other scheduled operations.

The platform uses **Spring Boot Scheduler**, **Asynchronous Processing**, and is designed for future migration to distributed schedulers such as **Quartz** or **Kubernetes CronJobs**.

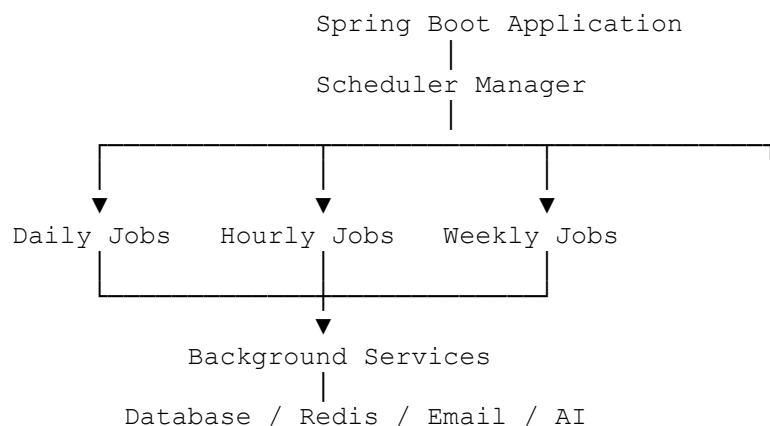
---

## 14.2 Objectives

The Scheduler Framework shall:

- Execute scheduled tasks automatically
  - Perform batch processing
  - Refresh cached data
  - Generate reports
  - Send reminders
  - Clean temporary data
  - Support retry and recovery
  - Enable future distributed scheduling
- 

## 14.3 High-Level Architecture



## 14.4 Scheduler Package Structure

```
scheduler
├── config
```

|   |           |
|---|-----------|
| — | jobs      |
| — | service   |
| — | listener  |
| — | monitor   |
| — | util      |
| — | constants |

---

## 14.5 Job Categories

The platform supports:

### Daily Jobs

- Reset Daily Missions
  - Update Daily Streak
  - Send Daily Challenge Reminder
  - Generate Daily Reports
- 

### Hourly Jobs

- Refresh Leaderboard
  - Refresh Dashboard Cache
  - Retry Failed Notifications
  - Clean Expired OTPs
- 

### Weekly Jobs

- Weekly Leaderboard
  - Weekly Analytics
  - Weekly Backup
  - Weekly Cleanup
- 

### Monthly Jobs

- Placement Statistics
  - Monthly Reports
  - Archive Logs
  - Backup Verification
-



## 14.6 Scheduler Flow

Application Starts

↓

Scheduler Registered

↓

Cron Trigger

↓

Execute Job

↓

Business Logic

↓

Update Database

↓

Log Result

↓

Complete

---

## 14.7 Spring Scheduler

Example

```
@Scheduled(cron = "0 0 0 * * *")
public void resetDailyMission(){

}
```

Scheduling Types

- Fixed Rate
  - Fixed Delay
  - Cron Expression
-

## 14.8 Daily Mission Scheduler

Runs every day.

Responsibilities

- Reset Daily Challenges
  - Generate New Missions
  - Reset Daily XP Limits
  - Update Streak Availability
- 

## 14.9 Leaderboard Scheduler

Runs every hour.

Updates

- Coding Leaderboard
- Aptitude Leaderboard
- SQL Leaderboard
- Overall Ranking

Redis cache is refreshed after calculation.

---

## 14.10 Notification Scheduler

Responsibilities

- Daily Reminder
- Interview Reminder
- Job Expiry Reminder
- Resume Reminder
- Streak Reminder

Channels

- Email
- SMS
- In-App

---

## 14.11 Report Scheduler

Generates

- Daily Reports
- Weekly Reports
- Monthly Reports

Reports

- Learner Activity
- Placement Reports
- Recruiter Reports
- Platform Usage

Generated files are stored securely for download.

---

## 14.12 Cache Refresh Scheduler

Refreshes

- Dashboard Cache
- Leaderboard Cache
- Job Cache
- AI Cache
- Configuration Cache

Purpose

- Faster response time
  - Reduced database load
- 

## 14.13 Database Cleanup Job

Removes

- Expired OTP

- Expired Tokens
- Temporary Files
- Expired Sessions
- Orphan Records

Runs during low-traffic hours.

---

## 14.14 Backup Scheduler

### Tasks

- Database Backup
- File Backup
- Configuration Backup
- Log Backup

### Schedules

- Daily Incremental
  - Weekly Full Backup
- 

## 14.15 AI Maintenance Jobs

### Tasks

- Clear AI Cache
  - Update Prompt Templates
  - Archive AI Conversations
  - Generate AI Usage Reports
- 

## 14.16 Batch Processing

Large operations are processed in batches.

### Examples

- Notification Delivery

- Email Campaigns
- Leaderboard Updates
- Report Generation
- Data Migration

#### Benefits

- Better performance
- Lower memory usage
- Improved reliability

---

## 14.17 Background Processing

Asynchronous tasks include

- Email Sending
- SMS Sending
- AI Processing
- PDF Generation
- Resume Analysis
- File Processing

#### Implementation

@Async

---

## 14.18 Retry Mechanism

#### Workflow

Job Starts

↓

Failure

↓

Retry

↓

Retry Again

↓

Mark Failed

↓

Notify Administrator

Configurable

- Maximum Retries
- Retry Interval
- Failure Threshold

---

## 14.19 Job Monitoring

Every scheduled job records:

- Job Name
- Start Time
- End Time
- Duration
- Status
- Records Processed
- Failure Reason

---

## 14.20 Job History

History includes

- Execution Time
- Result
- Retry Count
- Server Instance
- Error Log

Useful for troubleshooting and auditing.

---

## 14.21 Job Priority

### Priority Levels

- Critical
- High
- Medium
- Low

### Critical jobs

- Backup
  - Leaderboard
  - Notifications
- 

## 14.22 Error Handling

Scheduler errors are:

- Logged
- Retried (when appropriate)
- Alerted to administrators
- Stored in job history

Failures in one job must not stop other scheduled jobs.

---

## 14.23 Performance Optimization

Strategies

- Batch processing
  - Parallel execution
  - Asynchronous tasks
  - Redis caching
  - Pagination
  - Database indexing
- 

## 14.24 Future Distributed Scheduling

## Phase 2

### Support

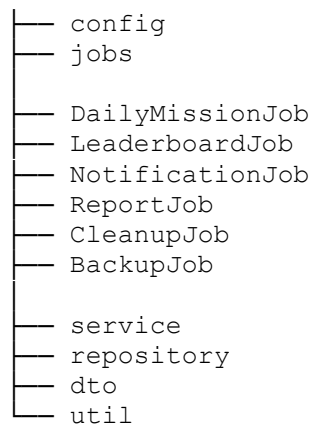
- Quartz Scheduler
- Kubernetes CronJobs
- Distributed Locks
- Multi-node Scheduling

This prevents duplicate execution in clustered deployments.

---

## 14.25 Package Structure

scheduler



## 14.26 Testing Strategy

Testing includes

- Scheduler Execution
- Cron Validation
- Retry Logic
- Batch Processing
- Cleanup Jobs
- Backup Jobs
- Failure Recovery
- Async Processing

Tools



- JUnit 5
  - Spring Boot Test
  - Mockito
- 

## 14.27 Best Practices

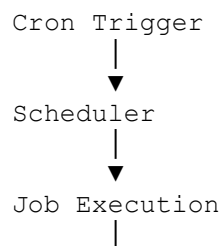
- Keep scheduled jobs idempotent.
  - Avoid long-running synchronous tasks.
  - Execute heavy jobs asynchronously.
  - Log every execution.
  - Implement retry with limits.
  - Monitor execution time.
  - Avoid overlapping jobs.
  - Store execution history.
  - Design jobs for horizontal scalability.
- 

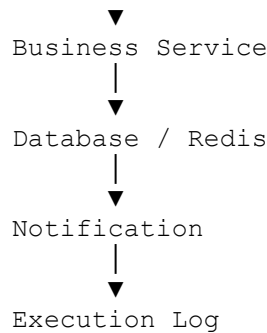
## 14.28 Acceptance Criteria

The Scheduler & Automation framework shall be accepted when:

- Scheduled jobs execute according to configured schedules.
  - Batch jobs process records correctly.
  - Background tasks run asynchronously where required.
  - Retry logic handles transient failures.
  - Job history is maintained.
  - Monitoring captures execution metrics.
  - Cleanup and backup jobs execute successfully.
  - Cache refresh and leaderboard updates complete without errors.
- 

## 14.29 Scheduler Workflow





## 14.30 Chapter Summary

The **Scheduler, Background Jobs, Batch Processing & System Automation** architecture provides a reliable automation framework for the Smart Education Platform. By leveraging Spring Scheduler, asynchronous execution, batch processing, monitoring, retry mechanisms, and structured job management, the platform automates operational tasks, improves system performance, reduces manual intervention, and prepares the application for future distributed scheduling in cloud-native environments.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 15 – Logging, Monitoring, Audit Trail & Observability Architecture

**Version:** 1.0

---

### 15.1 Introduction

The **Logging, Monitoring, Audit Trail & Observability Architecture** provides complete visibility into the Smart Education Platform by capturing application events, monitoring system health, tracking user activities, and generating operational insights.

The architecture enables developers, DevOps engineers, and administrators to quickly detect issues, investigate incidents, monitor performance, and maintain compliance through comprehensive audit records.

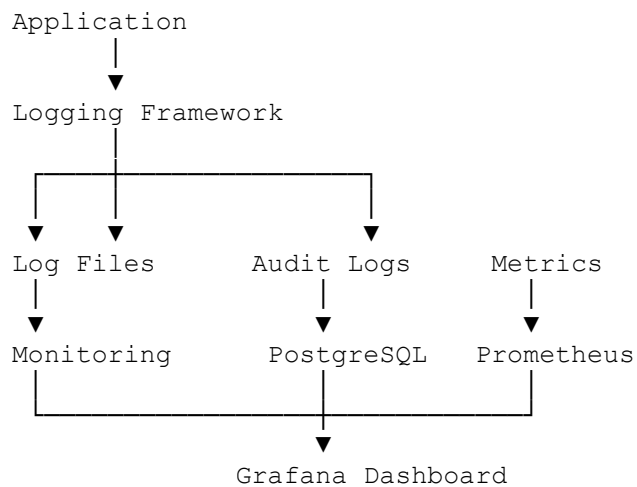
---

## 15.2 Objectives

The Logging & Monitoring framework shall:

- Record application events
  - Monitor application health
  - Track user activities
  - Audit administrative actions
  - Capture security events
  - Generate operational metrics
  - Support incident investigation
  - Improve production observability
- 

## 15.3 High-Level Architecture



## 15.4 Technology Stack

| Component     | Technology |
|---------------|------------|
| Logging       | SLF4J      |
| Log Framework | Logback    |

| Component          | Technology           |
|--------------------|----------------------|
| Monitoring         | Spring Boot Actuator |
| Metrics            | Micrometer           |
| Metrics Collection | Prometheus           |
| Dashboard          | Grafana              |
| Audit Storage      | PostgreSQL           |
| Log Aggregation    | ELK Stack (Future)   |

---

## 15.5 Logging Architecture

Controller

↓

Service

↓

Repository

↓

Logger

↓

Logback

↓

Log File

Every layer generates appropriate logs.

---

## 15.6 Log Levels

Supported Levels

| Level | Usage                     |
|-------|---------------------------|
| TRACE | Detailed debugging        |
| DEBUG | Development debugging     |
| INFO  | Normal application events |
| WARN  | Recoverable issues        |
| ERROR | Exceptions & failures     |

Production default:

INFO

---

## 15.7 Logging Categories

### Application Logs

- Startup
  - Shutdown
  - Configuration
  - API Requests
  - API Responses
- 

### Security Logs

- Login
  - Logout
  - Failed Login
  - Password Reset
  - JWT Validation Failure
  - Unauthorized Access
- 

### Business Logs

- Coding Submission
- SQL Submission

- Resume Upload
  - Job Application
  - Interview Scheduled
  - Badge Earned
- 

### System Logs

- Cache Refresh
  - Scheduler
  - Database Backup
  - Cleanup Job
  - AI Requests
- 

## 15.8 Structured Logging

Each log entry contains

- Timestamp
- Request ID
- User ID
- Module
- Log Level
- Message
- Execution Time
- Server Instance

Example

2026-07-10 10:30:15

INFO

User=1001

Module=Coding

RequestId=ABC123

Message=Submission Successful

---

## 15.9 Audit Trail

Every important action is stored.

#### Audit Fields

- Audit ID
  - User ID
  - User Role
  - Module
  - Action
  - Previous Value
  - New Value
  - Timestamp
  - IP Address
  - Device Information
- 

## 15.10 Audit Events

#### Learner

- Profile Updated
  - Resume Uploaded
  - Coding Submitted
  - Job Applied
- 

#### Recruiter

- Job Created
  - Job Updated
  - Interview Scheduled
  - Offer Released
- 

#### Super Admin

- User Created
  - User Blocked
  - Recruiter Approved
  - Question Published
  - System Configuration Changed
-

## 15.11 Audit Flow

User Action

↓

Business Service

↓

Audit Service

↓

Audit Repository

↓

Audit Table

---

## 15.12 Request Correlation

Each incoming request receives a unique **Correlation ID (Request ID)**.

Flow

Request

↓

Correlation ID

↓

Controller

↓

Service

↓

Repository

↓

Logs

This allows complete request tracing.



---

## 15.13 Monitoring Architecture

### Monitored Components

- Spring Boot
- PostgreSQL
- Redis
- AI Services
- Scheduler
- Storage
- SMTP
- Notification Service

---

## 15.14 Health Checks

### Spring Boot Actuator Endpoints

`/actuator/health`

`/actuator/info`

`/actuator/metrics`

`/actuator/prometheus`

---

## 15.15 Metrics Collection

### Application Metrics

- API Response Time
- API Count
- Success Rate
- Error Rate
- Active Users
- Login Count

### Infrastructure Metrics

- CPU
- Memory

- Disk
  - JVM Heap
  - Database Connections
- 

## 15.16 Prometheus Integration

Prometheus collects

- JVM Metrics
- HTTP Metrics
- Database Metrics
- Redis Metrics
- Scheduler Metrics
- Custom Business Metrics

Scrape Interval

- 15 Seconds
- 

## 15.17 Grafana Dashboards

Dashboards

### Application

- Request Rate
  - API Latency
  - Error Rate
- 

### Infrastructure

- CPU
  - Memory
  - Disk
  - JVM
-

## Business

- Active Learners
  - Coding Attempts
  - Job Applications
  - Recruiter Activity
- 

## AI

- AI Requests
  - Token Usage
  - Response Time
  - Provider Availability
- 

# 15.18 Alerting Strategy

## Alerts

- High CPU Usage
- Low Memory
- High Error Rate
- Database Down
- Redis Down
- AI Service Failure
- SMTP Failure
- Backup Failure

Alerts sent to administrators via Email and SMS.

---

# 15.19 Performance Monitoring

## Track

- Dashboard Load Time
- API Response Time
- SQL Query Duration
- Cache Hit Ratio
- AI Response Time

Target

| Metric | Target |
|--------|--------|
|--------|--------|

|              |         |
|--------------|---------|
| API Response | <500 ms |
|--------------|---------|

|           |        |
|-----------|--------|
| Dashboard | <2 sec |
|-----------|--------|

|             |        |
|-------------|--------|
| AI Response | <5 sec |
|-------------|--------|

---

## 15.20 Security Monitoring

Monitor

- Failed Login Attempts
  - JWT Validation Failures
  - Access Denied Requests
  - Brute Force Attempts
  - Unusual Login Patterns
  - Multiple Password Reset Requests
- 

## 15.21 Log Retention Policy

| Log Type | Retention |
|----------|-----------|
|----------|-----------|

|                  |         |
|------------------|---------|
| Application Logs | 90 Days |
|------------------|---------|

|            |         |
|------------|---------|
| Audit Logs | 7 Years |
|------------|---------|

|               |        |
|---------------|--------|
| Security Logs | 1 Year |
|---------------|--------|

|                |          |
|----------------|----------|
| Scheduler Logs | 180 Days |
|----------------|----------|

|            |        |
|------------|--------|
| Error Logs | 1 Year |
|------------|--------|

---

## 15.22 Log Rotation

## Logback Configuration

- Daily Rotation
- File Compression
- Maximum File Size
- Automatic Cleanup

This prevents excessive disk usage.

---

## 15.23 Exception Logging

All exceptions include

- Exception Type
- Message
- Stack Trace (Internal Only)
- Request ID
- User ID
- API Endpoint

Stack traces must never be exposed to API clients.

---

## 15.24 Package Structure

logging

```
|— config
|— audit
|— monitor
|— metrics
|— listener
|— interceptor
|— filter
|— repository
|— dto
|— util
```

---

## 15.25 Distributed Tracing (Future)

Future support

- OpenTelemetry
- Jaeger
- Zipkin

#### Benefits

- End-to-end request tracing
- Service dependency visualization
- Latency analysis

---

## 15.26 Testing Strategy

#### Testing includes

- Log Generation
- Audit Creation
- Health Endpoint Validation
- Metrics Collection
- Alert Triggering
- Dashboard Accuracy
- Log Rotation
- Monitoring Integration

#### Tools

- JUnit 5
- Spring Boot Test
- MockMvc
- Prometheus Test Environment

---

## 15.27 Best Practices

- Use structured logging.
- Never log passwords, OTPs, JWTs, or sensitive personal data.
- Generate a correlation ID for every request.
- Separate application, audit, and security logs.
- Rotate logs regularly.
- Monitor business and infrastructure metrics.
- Configure alerts for critical failures.
- Retain logs according to compliance requirements.

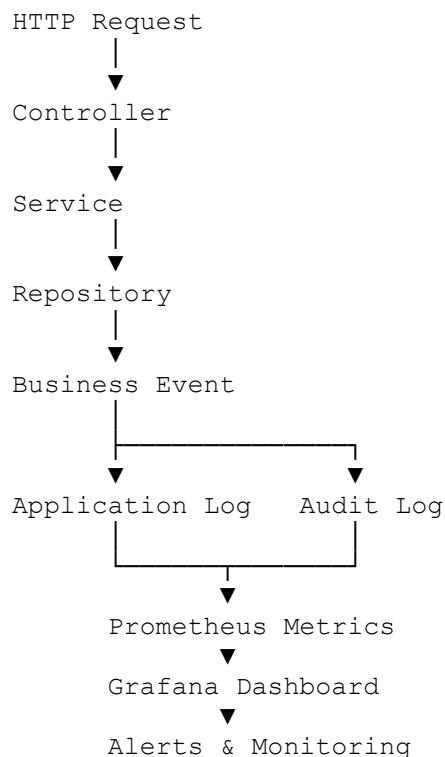
---

## 15.28 Acceptance Criteria

The Logging & Observability framework shall be accepted when:

- Application logs are generated consistently.
  - Audit trails capture all critical business actions.
  - Health endpoints report application status correctly.
  - Prometheus collects application metrics.
  - Grafana dashboards display real-time monitoring data.
  - Alerts are triggered for configured failure conditions.
  - Log rotation and retention policies function correctly.
  - Sensitive information is excluded from logs.
- 

## 15.29 Complete Observability Flow



## 15.30 Chapter Summary

The **Logging, Monitoring, Audit Trail & Observability Architecture** provides comprehensive operational visibility for the Smart Education Platform. By combining structured logging, centralized audit trails, application and infrastructure monitoring, metrics collection, health checks, alerting, and future-ready distributed tracing, the platform ensures high reliability, easier troubleshooting, security compliance, and efficient production operations.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 16 – Complete Sequence Diagrams, Class Diagrams & Module Integration Design

**Version:** 1.0

---

### 16.1 Introduction

This chapter defines the **UML design** of the Smart Education Platform. It explains how different modules interact internally and illustrates the flow of requests using **Sequence Diagrams, Class Diagrams, Activity Diagrams, and Module Interaction Diagrams**.

This design serves as the blueprint for developers during implementation.

---

### 16.2 Objectives

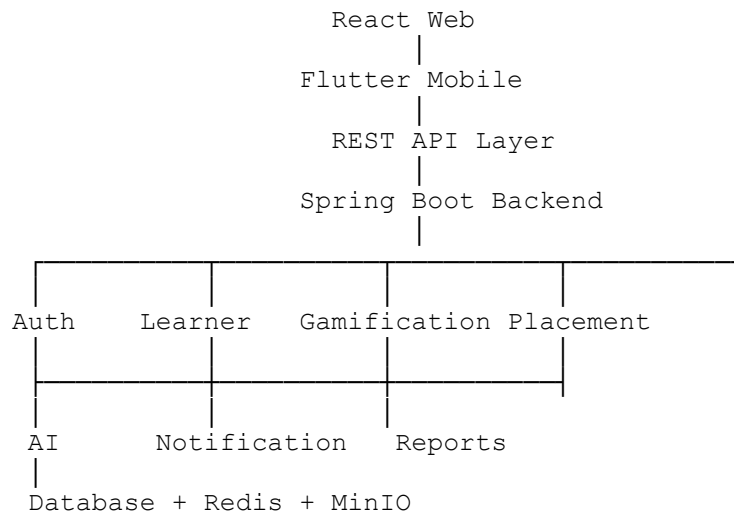
The UML design shall:

- Describe object interactions
- Define class relationships
- Explain module communication
- Standardize business workflows
- Improve maintainability
- Reduce implementation ambiguity



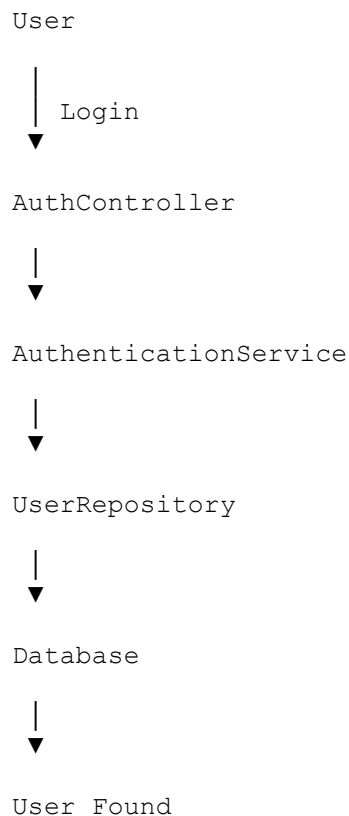
---

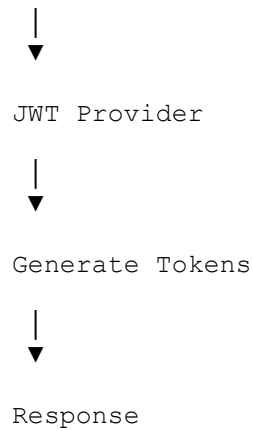
## 16.3 Complete System Architecture



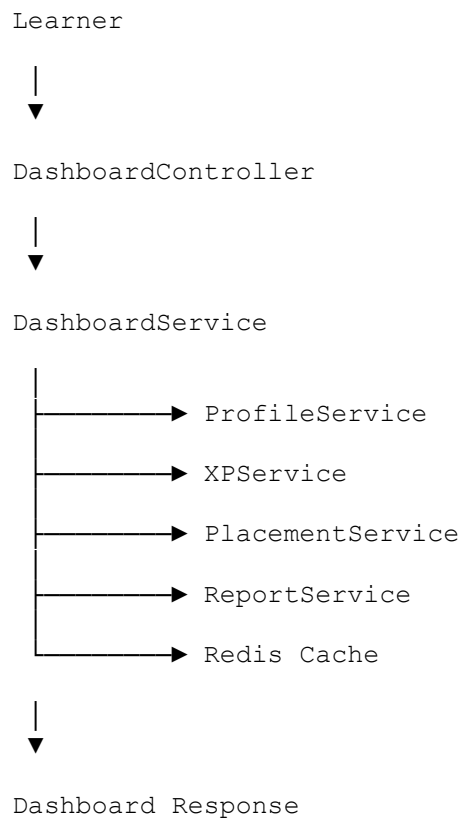
---

## 16.4 Authentication Sequence Diagram



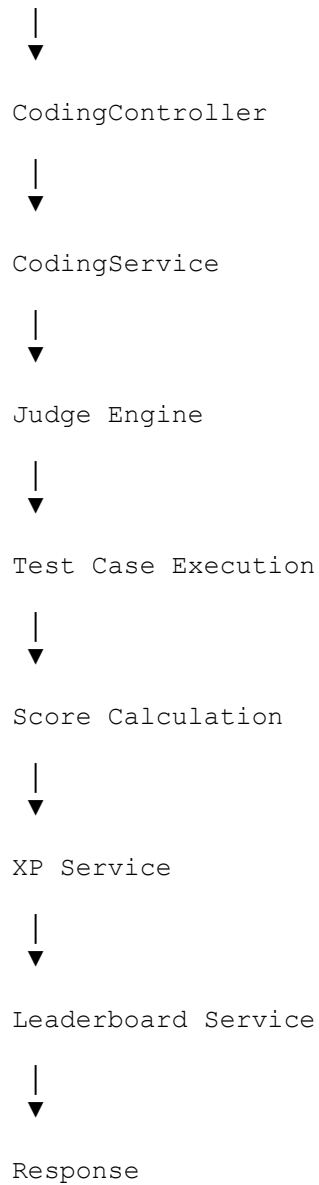


## 16.5 Learner Dashboard Sequence

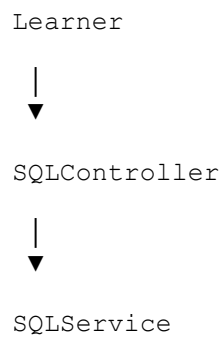


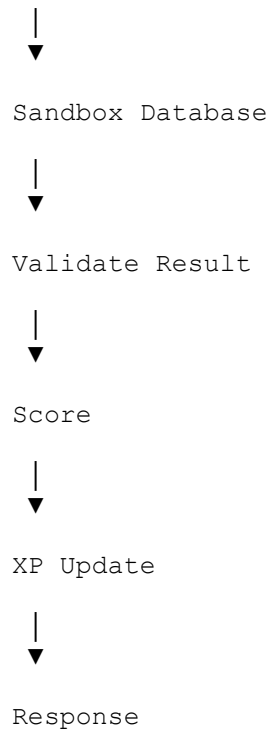
## 16.6 Coding Arena Sequence Diagram

Learner



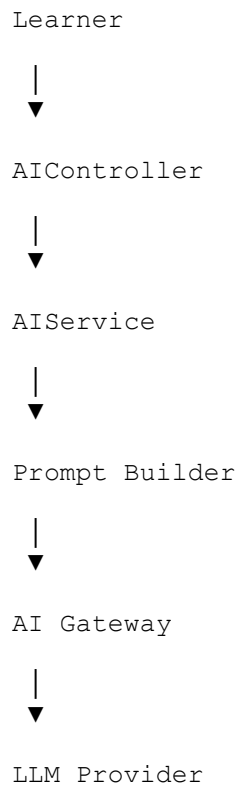
## 16.7 SQL Arena Sequence Diagram





---

## 16.8 AI Coach Sequence Diagram





AI Response



Conversation Repository



Return Response

---

## 16.9 Resume Builder Sequence

Learner



ResumeController



ResumeService



Resume Repository



ATS AI Review



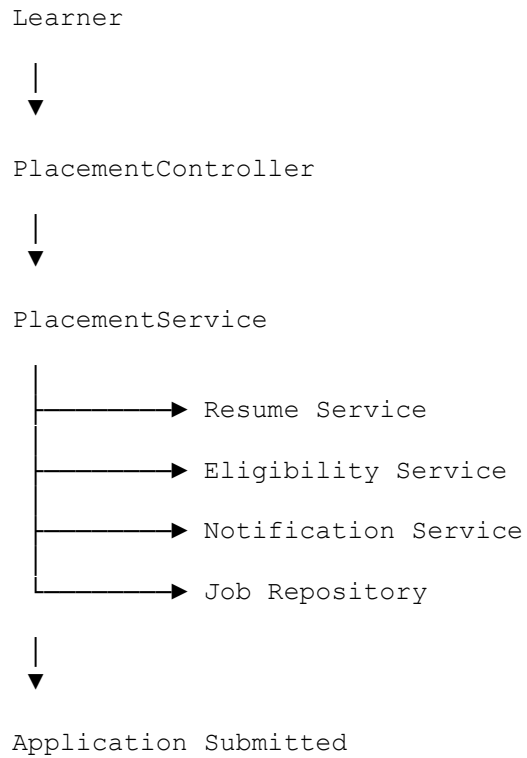
Resume Score



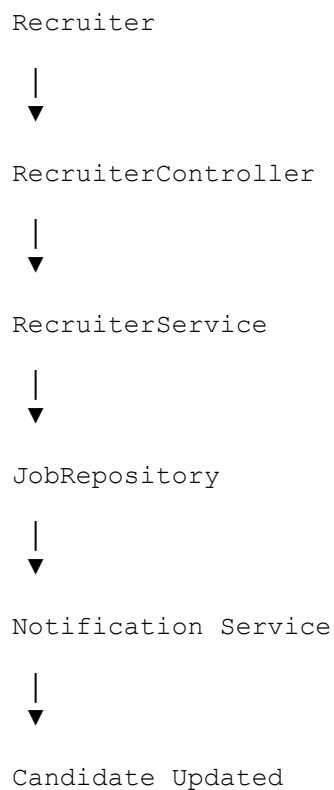
Return Resume

---

## 16.10 Placement Portal Sequence



## 16.11 Recruiter Workflow



---

## 16.12 Super Admin Workflow

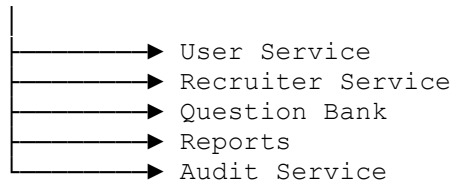
Admin



AdminController



AdminService



Success

---

## 16.13 Notification Sequence

Business Event



Notification Service



Template Engine



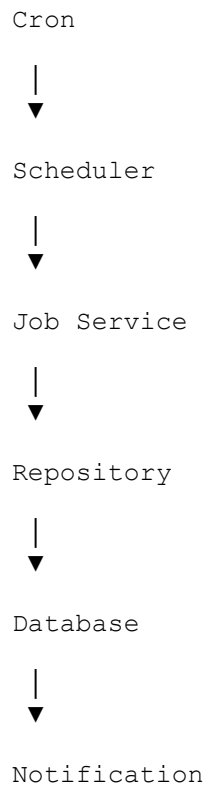
Email / SMS / In-App



Notification History

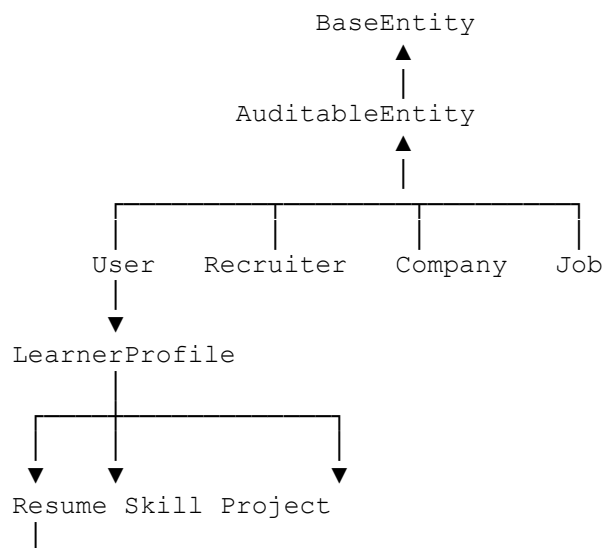
---

## 16.14 Scheduler Sequence

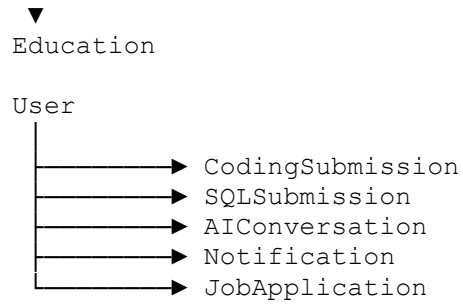


---

## 16.15 Complete Class Diagram







## 16.16 Authentication Class Diagram

AuthenticationController

↓

AuthenticationService

↓

AuthenticationProvider

↓

JWTProvider

↓

UserRepository

↓

User Entity

---

## 16.17 Coding Module Class Diagram

CodingController

↓

CodingService

↓

CodingRepository

↓

CodingProblem

↓

CodingSubmission

↓

CodingTestCase

---

## 16.18 Placement Class Diagram

PlacementController

↓

PlacementService

↓

JobRepository

↓

Job

↓

Application

↓

Interview

↓

Offer

---

## 16.19 Recruiter Class Diagram

Recruiter

↓

Company

↓

Job



Interview



Offer

---

## 16.20 AI Module Class Diagram

AIController



AIService



PromptBuilder



AIGateway



AIProvider



ConversationRepository

---

## 16.21 Notification Class Diagram

NotificationController



NotificationService



EmailProvider



SMSProvider

↓

NotificationRepository

---

## 16.22 Activity Diagram – Job Application

Start

↓

View Jobs

↓

Select Job

↓

Eligibility Check

↓

Resume Available?

↓

Yes

↓

Apply

↓

Save Application

↓

Send Notification

↓

End

---

## 16.23 Activity Diagram – Coding Challenge

Start

↓

Choose Problem



Write Code



Submit



Compile



Run Test Cases



Passed?



Calculate XP



Update Leaderboard



Finish

---

## 16.24 Module Integration Diagram

Authentication



Learner



Gamification



Brain Gym

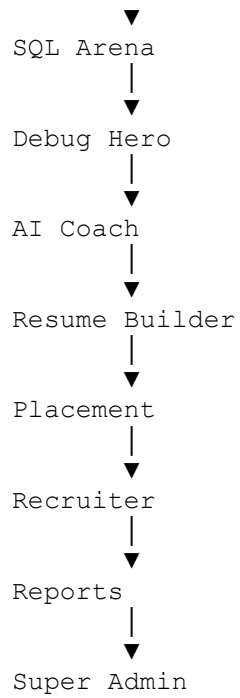


Aptitude

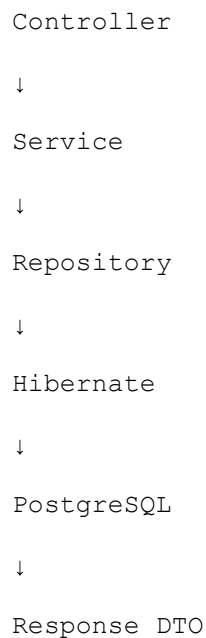


Coding Arena

↓



## 16.25 Database Interaction Diagram



## 16.26 External Service Integration

Application

└─ AI Provider

└─ SMTP Server

└─ SMS Gateway

└─ Redis

└─ MinIO

└─ Monitoring Server

---

## 16.27 Request Lifecycle Diagram

HTTP Request

↓

Security Filter

↓

Controller

↓

Validation

↓

Service

↓

Repository

↓

Database

↓

Mapper

↓

Response DTO

↓

HTTP Response

---

## 16.28 Design Principles

The UML design follows these principles:

- Layered Architecture
- SOLID Principles
- Dependency Injection
- Single Responsibility Principle
- Interface-based Programming
- Low Coupling
- High Cohesion
- Domain-Driven Design (DDD)

---

## 16.29 Best Practices

- Keep controllers independent of business logic.
- Maintain one responsibility per class.
- Use interfaces for all services.
- Avoid circular dependencies.
- Design reusable domain models.
- Use DTOs instead of exposing entities.
- Keep module interactions loosely coupled.
- Document sequence flows for critical business processes.

---

## 16.30 Acceptance Criteria

The UML and module integration design shall be accepted when:

- Sequence diagrams accurately represent all critical workflows.
  - Class diagrams reflect the implemented domain model.
  - Module interactions are clearly defined.
  - Controller–Service–Repository flow is consistently followed.
  - External service integrations are documented.
  - Activity diagrams cover major user journeys.
  - The design aligns with the platform's layered architecture and coding standards.
-



# 16.31 Chapter Summary

The **Complete Sequence Diagrams, Class Diagrams & Module Integration Design** provides a comprehensive blueprint of how the Smart Education Platform operates internally. It defines interactions between components, models domain relationships, illustrates end-to-end business workflows, and documents module integrations, enabling developers to build a consistent, scalable, and maintainable application architecture.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 17 – Design Patterns, SOLID Principles, Clean Architecture & Coding Standards

**Version:** 1.0

---

### 17.1 Introduction

This chapter defines the software engineering principles and architectural standards used throughout the Smart Education Platform. It ensures that the codebase remains scalable, maintainable, testable, secure, and easy to extend as new modules are introduced.

The platform follows:

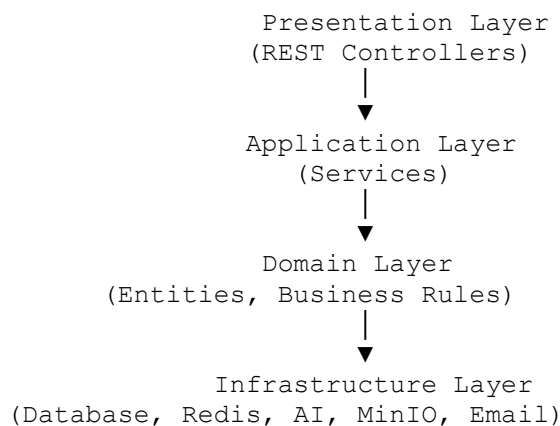
- SOLID Principles
  - Clean Architecture
  - Domain-Driven Design (DDD)
  - Layered Architecture
  - Object-Oriented Programming (OOP)
  - GoF (Gang of Four) Design Patterns
  - Secure Coding Standards
  - Java & Spring Boot Best Practices
-

## 17.2 Objectives

The architecture shall:

- Produce clean and maintainable code
  - Reduce coupling
  - Increase cohesion
  - Promote code reusability
  - Simplify testing
  - Improve scalability
  - Support future microservices migration
- 

## 17.3 Clean Architecture



Dependency Rule

- Outer layers depend on inner layers.
  - Domain Layer never depends on Infrastructure.
- 

## 17.4 Layer Responsibilities

### Presentation Layer

Responsibilities

- REST APIs

- Request Validation
- Response Formatting
- Authentication

Contains

- Controllers
  - DTOs
- 

## Application Layer

Responsibilities

- Business Logic
- Transaction Management
- Module Coordination

Contains

- Services
  - Mappers
- 

## Domain Layer

Responsibilities

- Business Entities
- Domain Rules
- Value Objects
- Enumerations

Contains

- Entities
  - Interfaces
  - Domain Events
- 

## Infrastructure Layer

## Responsibilities

- Database
- Redis
- Email
- AI Integration
- MinIO
- External APIs

---

# 17.5 SOLID Principles

## S — Single Responsibility Principle (SRP)

Each class should have only one reason to change.

### Good Example

```
ResumeService
```

```
Only manages resumes.
```

### Avoid

```
ResumeService
```

```
Creates Resume
```

```
Sends Email
```

```
Generates PDF
```

```
Updates Leaderboard
```

---

## O — Open/Closed Principle (OCP)

Software should be:

- Open for Extension
- Closed for Modification

### Example

```
NotificationService
```

↓

EmailProvider

SMSProvider

PushProvider

Adding WhatsApp notifications requires a new provider implementation without modifying existing code.

---

## L — Liskov Substitution Principle (LSP)

Child classes must be replaceable by parent classes.

Example

AIProvider

↓

OpenAIProvider

AzureAIProvider

Both providers can be used interchangeably.

---

## I — Interface Segregation Principle (ISP)

Prefer small, focused interfaces.

Good

ResumeService

CodingService

PlacementService

Avoid

PlatformService

containing hundreds of unrelated methods.

---

## D — Dependency Inversion Principle (DIP)

Depend on abstractions.

Good

Controller

↓

Service Interface

↓

Service Implementation

Avoid direct dependencies on concrete implementations.

---

## 17.6 Dependency Injection

Spring manages dependencies.

Example

```
@Service
public class CodingServiceImpl
implements CodingService {

    private final CodingRepository repository;

    public CodingServiceImpl(
        CodingRepository repository) {

        this.repository = repository;
    }
}
```

Use constructor injection throughout the application.

---

## 17.7 Domain-Driven Design (DDD)

Modules are organized around business domains.

Authentication

Learner

Gamification

Brain Gym

Aptitude

Coding Arena

SQL Arena

Debug Hero

AI Coach

Resume Builder

Placement

Recruiter

Reports

Notification

Each module owns its entities, services, repositories, and DTOs.

---

## 17.8 Design Patterns Used

| Pattern         | Purpose                  |
|-----------------|--------------------------|
| Singleton       | Configuration Beans      |
| Factory         | AI Provider Creation     |
| Strategy        | Scoring Algorithms       |
| Builder         | DTO Construction         |
| Adapter         | External API Integration |
| Observer        | Event Notifications      |
| Template Method | Assessment Workflow      |
| Facade          | AI Gateway               |
| Proxy           | Security Layer           |
| Command         | Scheduler Tasks          |

---

## 17.9 Factory Pattern

Used for

- AI Providers
- Notification Providers
- File Storage Providers

Example

AIFactory

↓

OpenAI

Azure OpenAI

Future LLM

---

## 17.10 Strategy Pattern

Used for

- Coding Score Calculation
- SQL Evaluation
- Placement Score
- Badge Calculation
- XP Calculation

Example

ScoreStrategy

↓

CodingScore

SQLScore

AptitudeScore

---

## 17.11 Builder Pattern

Used for



- Response DTO
- Resume Generation
- AI Prompt Construction

#### Benefits

- Readable object creation
- Immutable objects
- Flexible configuration

---

## 17.12 Observer Pattern

#### Used for

- Notification System
- Audit Logging
- Leaderboard Updates

#### Example

Coding Completed

↓

XP Updated

↓

Notification

↓

Report Updated

---

## 17.13 Template Method Pattern

#### Used in assessments.

Start Assessment

↓

Load Questions

↓

Submit Answers

↓

Evaluate

↓

Calculate Score

↓

Generate Report

The common workflow remains fixed while module-specific evaluation logic varies.

---

## 17.14 Adapter Pattern

Used for external services.

Examples

- SMTP Provider
- SMS Provider
- AI Provider
- Cloud Storage

Allows changing providers with minimal impact on business logic.

---

## 17.15 Facade Pattern

Example

AIFacade

↓

Prompt Builder

↓

AI Provider

↓

Response Parser

The facade simplifies AI interactions for other modules.

---

## 17.16 Coding Standards

### Java Naming

#### Classes

ResumeService

CodingController

PlacementRepository

#### Variables

userName

placementScore

currentLevel

#### Constants

MAX\_FILE\_SIZE

TOKEN\_EXPIRY

DEFAULT\_PAGE\_SIZE

#### Packages

controller

service

repository

entity

dto

mapper

config

---

## 17.17 Exception Handling Standards

- Use custom exceptions.
  - Do not expose stack traces.
  - Use centralized exception handling.
  - Return standardized API responses.
- 

## 17.18 Logging Standards

### Log

- Business Events
- Errors
- Warnings
- Security Events

### Do Not Log

- Passwords
  - OTPs
  - JWT Tokens
  - API Keys
  - Sensitive personal data
- 

## 17.19 Security Coding Guidelines

- Validate all inputs.
  - Use parameterized queries.
  - Encrypt passwords using BCrypt.
  - Sanitize uploaded files.
  - Protect against SQL Injection.
  - Protect against XSS.
  - Protect against CSRF where applicable.
  - Use HTTPS for all communication.
- 

## 17.20 Performance Coding Standards

- Prefer constructor injection.
  - Avoid unnecessary object creation.
  - Cache expensive operations.
  - Use pagination.
  - Use lazy loading where appropriate.
  - Avoid N+1 query problems.
  - Use asynchronous processing for long-running tasks.
- 

## 17.21 Code Review Checklist

Review for:

- SOLID compliance
  - Naming conventions
  - Error handling
  - Logging
  - Security
  - Performance
  - Unit tests
  - Transaction management
  - API documentation
  - Code readability
- 

## 17.22 Package Organization

`com.smarteducation`

```
|— authentication
|— learner
|— gamification
|— braingym
|— aptitude
|— coding
|— sql
|— debughero
|— aicoach
|— resume
|— placement
|— recruiter
|— notification
|— reports
|— common
|— config
```

---

## 17.23 Code Quality Tools

Recommended tools:

- SonarQube
- Checkstyle
- PMD
- SpotBugs
- JaCoCo
- Maven Enforcer Plugin

Quality Gates

- No blocker issues
  - No critical security vulnerabilities
  - Code coverage  $\geq 80\%$
  - Minimal code duplication
- 

## 17.24 Documentation Standards

Every public API and service should include:

- JavaDoc
- Method description
- Parameter description
- Return value
- Exception details

REST APIs should be documented using OpenAPI (Swagger).

---

## 17.25 Best Practices

- Keep methods short and focused.
- Follow the Single Responsibility Principle.
- Program to interfaces.
- Use immutable objects where practical.
- Avoid duplicated logic.
- Keep controllers lightweight.
- Use descriptive names.

- Write meaningful unit tests.
  - Refactor regularly.
  - Document complex business rules.
- 

## 17.26 Acceptance Criteria

The design and coding standards shall be accepted when:

- SOLID principles are consistently applied.
  - Clean Architecture boundaries are maintained.
  - Appropriate design patterns are used.
  - Coding standards are followed across all modules.
  - Security and performance guidelines are implemented.
  - Code quality tools pass configured quality gates.
  - Code reviews confirm adherence to architectural standards.
- 

## 17.27 Chapter Summary

The **Design Patterns, SOLID Principles, Clean Architecture & Coding Standards** establish the engineering foundation of the Smart Education Platform. By combining modular architecture, domain-driven design, proven design patterns, secure coding practices, and consistent development standards, the platform achieves high maintainability, extensibility, and long-term scalability while ensuring clean, testable, and production-ready code.

## Low-Level Design (LLD)

## Smart Education Platform (Phase 1)

## Chapter 18 – Unit Testing, Integration Testing, API Testing & Test Automation Framework

Version: 1.0

---

# 18.1 Introduction

Testing is a critical part of the Smart Education Platform to ensure reliability, security, scalability, and maintainability. This chapter defines the complete testing architecture, testing standards, automation framework, and quality assurance strategy for all platform modules.

The testing strategy follows the **Testing Pyramid**, emphasizing a high volume of unit tests, supported by integration tests and end-to-end API tests.

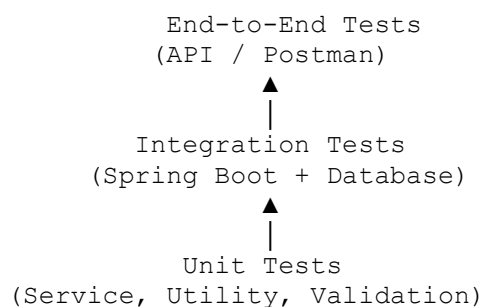
---

## 18.2 Objectives

The Test Automation Framework shall:

- Validate all business logic
  - Ensure API correctness
  - Verify database integration
  - Prevent regressions
  - Automate testing in CI/CD
  - Improve code quality
  - Increase deployment confidence
- 

## 18.3 Testing Architecture



## 18.4 Testing Strategy

The platform follows multiple testing levels:

- Unit Testing



- Integration Testing
- Repository Testing
- Controller Testing
- API Testing
- Security Testing
- Performance Testing
- Load Testing
- Regression Testing
- User Acceptance Testing (UAT)

---

## 18.5 Testing Tools

| Category            | Tool             |
|---------------------|------------------|
| Unit Testing        | JUnit 5          |
| Mocking             | Mockito          |
| Controller Testing  | MockMvc          |
| Integration Testing | Spring Boot Test |
| Database Testing    | Testcontainers   |
| API Testing         | Postman          |
| API Automation      | Newman           |
| Code Coverage       | JaCoCo           |
| Load Testing        | JMeter           |
| Security Testing    | OWASP ZAP        |

---

## 18.6 Project Test Structure

```
src
```

```
|— test
```

```
|— java
```

```
|
| |— authentication
| |— learner
| |— gamification
| |— coding
| |— sqlarena
| |— placement
| |— recruiter
| |— admin
| |— common
```

## 18.7 Unit Testing

Unit tests verify individual classes in isolation.

Tested Components

- Services
- Utilities
- Validators
- Mappers
- Helper Classes

Dependencies are mocked using Mockito.

---

## 18.8 Unit Test Example

```
@ExtendWith(MockitoExtension.class)
class CodingServiceTest {

    @Mock
    private CodingRepository repository;

    @InjectMocks
    private CodingServiceImpl service;

}
```

---

## 18.9 Mockito Usage

Mockito is used for

- Mock Repository
- Mock AI Provider
- Mock Email Service
- Mock Notification Service
- Mock Redis

Benefits

- Fast execution
  - Independent tests
  - No external dependencies
- 

## 18.10 Repository Testing

Repositories are tested using:

- Testcontainers
- PostgreSQL
- Spring Data JPA

Verify

- CRUD
  - JPQL Queries
  - Specifications
  - Pagination
  - Sorting
- 

## 18.11 Controller Testing

Uses

`MockMvc`

Tests

- Request Validation
- Authentication
- Authorization
- Response Codes
- Response Body

No real server required.

---

## 18.12 Integration Testing

Integration testing verifies communication between

- Controller
- Service
- Repository
- Database

Uses

@SpringBootTest

---

## 18.13 Testcontainers

Real PostgreSQL container.

Benefits

- Real Database
- Reliable Tests
- Environment Consistency
- Production-like Testing

Future support

- Redis Container
  - MinIO Container
- 

## 18.14 API Testing

All REST APIs tested.

Tools

- Postman
- Newman

Tests

- Success
- Validation Failure
- Authentication

- Authorization
  - Error Handling
  - Pagination
- 

## 18.15 Authentication Testing

Verify

- Login
  - Logout
  - Refresh Token
  - JWT Validation
  - Password Reset
  - Invalid Credentials
  - Expired Token
- 

## 18.16 Learner Testing

Verify

- Profile Update
  - Resume Upload
  - Dashboard
  - XP
  - Leaderboard
- 

## 18.17 Coding Arena Testing

Verify

- Problem Fetch
  - Submission
  - Compilation Result
  - XP Calculation
  - Ranking Update
-

## 18.18 SQL Arena Testing

Verify

- Query Execution
  - Validation
  - Submission
  - Score Calculation
- 

## 18.19 AI Testing

Verify

- Prompt Generation
- AI Gateway
- Retry Logic
- Timeout Handling
- Response Validation
- Token Tracking

External AI providers are mocked during automated tests.

---

## 18.20 Placement Testing

Verify

- Job Search
  - Job Application
  - Eligibility
  - Offer Acceptance
- 

## 18.21 Recruiter Testing

Verify

- Company Registration

- Job Posting
  - Candidate Search
  - Interview Scheduling
- 

## 18.22 Admin Testing

Verify

- User Management
  - Question Bank
  - Reports
  - Configuration
- 

## 18.23 Security Testing

Verify

- JWT Authentication
  - Role Validation
  - Permission Validation
  - CSRF Configuration
  - SQL Injection Protection
  - XSS Protection
- 

## 18.24 Performance Testing

Measure

- API Response Time
  - Dashboard Loading
  - Database Query Time
  - Redis Cache Performance
  - AI Response Time
-

## 18.25 Load Testing

Using Apache JMeter.

Scenarios

- 100 Concurrent Users
- 500 Concurrent Users
- 1000 Concurrent Users

Measure

- Throughput
  - Error Rate
  - Average Response Time
- 

## 18.26 Code Coverage

Tool

JaCoCo

Target Coverage

| Layer           | Coverage |
|-----------------|----------|
| Service         | 90%      |
| Controller      | 80%      |
| Repository      | 80%      |
| Utility         | 95%      |
| Overall Project | 85%+     |

---

## 18.27 CI/CD Testing Pipeline

Git Push

↓

Build

↓



Unit Tests

↓

Integration Tests

↓

API Tests

↓

Code Coverage

↓

Quality Check

↓

Docker Build

↓

Deploy

Deployment proceeds only if all mandatory quality gates pass.

---

## 18.28 Test Data Management

Separate datasets

- Unit Test Data
- Integration Test Data
- Performance Test Data
- UAT Data

Never use production data directly.

---

## 18.29 Test Reports

Generated Reports

- JUnit Report

- JaCoCo Coverage
  - Postman Report
  - Newman Report
  - JMeter Report
- 

## 18.30 Regression Testing

Executed

- Every Release
- Every Sprint
- Before Production Deployment

Modules Covered

- Authentication
  - Learner
  - Gamification
  - Placement
  - Recruiter
  - Admin
- 

## 18.31 Automation Framework

JUnit

↓

Mockito

↓

Spring Boot Test

↓

MockMvc

↓

Testcontainers

↓

Postman

↓

Newman

↓

CI/CD

---

## 18.32 Package Structure

test

```
|— unit
|— integration
|— api
|— performance
|— security
|— common
└— util
```

---

## 18.33 Best Practices

- Write tests for every business rule.
  - Mock external services.
  - Keep unit tests fast.
  - Use real databases for integration tests.
  - Automate API testing.
  - Maintain high code coverage.
  - Execute tests in CI/CD pipelines.
  - Use meaningful test names.
  - Keep tests independent and repeatable.
- 

## 18.34 Acceptance Criteria

The Test Automation Framework shall be accepted when:

- Unit tests achieve the required coverage targets.
  - Integration tests validate module interactions successfully.
  - Repository and controller tests pass consistently.
  - API automation verifies all critical endpoints.
  - Security and authentication scenarios are covered.
  - Performance and load tests meet defined benchmarks.
  - CI/CD pipelines execute automated tests successfully.
  - Test reports are generated for every build.
- 

## 18.35 Testing Workflow

Developer Code

↓

Unit Tests

↓

Integration Tests

↓

API Tests

↓

Security Tests

↓

Performance Tests

↓

Quality Report

↓

Deployment

---

## 18.36 Chapter Summary

The **Unit Testing, Integration Testing, API Testing & Test Automation Framework** provides a comprehensive quality assurance strategy for the Smart Education Platform. By combining automated unit tests, integration tests, API validation, security verification, performance testing, and CI/CD integration, the framework ensures that every release is reliable,

secure, maintainable, and production-ready while supporting continuous delivery and long-term scalability.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 19 – CI/CD Pipeline, DevOps Workflow & Release Management

Version: 1.0

---

### 19.1 Introduction

The **CI/CD (Continuous Integration & Continuous Deployment)** architecture automates the complete software delivery lifecycle of the Smart Education Platform. It enables developers to build, test, package, deploy, and monitor applications with minimal manual intervention while ensuring quality, reliability, and faster release cycles.

The DevOps workflow supports:

- Automated Build
  - Automated Testing
  - Code Quality Analysis
  - Docker Image Creation
  - Environment-based Deployment
  - Production Release
  - Rollback Strategy
  - Continuous Monitoring
- 

### 19.2 Objectives

The DevOps framework shall:

- Automate software delivery
- Reduce deployment failures

- Improve release speed
  - Ensure code quality
  - Enable zero/minimal downtime deployments
  - Support multiple deployment environments
  - Simplify rollback procedures
- 

## 19.3 DevOps Architecture

Developer

↓

Git Repository

↓

GitHub Actions / Jenkins

↓

Build

↓

Unit Tests

↓

Integration Tests

↓

SonarQube

↓

Docker Build

↓

Docker Registry

↓

Deployment

↓

Production Server

↓

## 19.4 Technology Stack

| Component                                    | Technology               |
|----------------------------------------------|--------------------------|
| Source Control                               | Git                      |
| Repository                                   | GitHub                   |
| CI/CD                                        | GitHub Actions / Jenkins |
| Build Tool                                   | Maven                    |
| Containerization                             | Docker                   |
| Reverse Proxy                                | Nginx                    |
| Monitoring                                   | Prometheus + Grafana     |
| Log Management                               | ELK Stack (Future)       |
| Container Orchestration Kubernetes (Phase 2) |                          |

---

## 19.5 Git Branching Strategy

main

|

production

|

develop

|

feature/\*

|

bugfix/\*

|

hotfix/\*

### Branch Purpose

- **main** – Stable production-ready code
  - **production** – Live deployment branch
  - **develop** – Integration branch
  - **feature/** – New feature development
  - **bugfix/** – Bug fixes
  - **hotfix/** – Critical production fixes
- 

## 19.6 Development Workflow

Developer

↓

Create Feature Branch

↓

Code Development

↓

Commit

↓

Push

↓

Pull Request

↓

Code Review

↓

Merge into Develop

↓

CI Pipeline

↓



## 19.7 CI Pipeline

### Pipeline Steps

Git Push

↓

Checkout Code

↓

Maven Build

↓

Unit Tests

↓

Integration Tests

↓

Code Coverage

↓

SonarQube Analysis

↓

Docker Build

↓

Push Docker Image

↓

Deploy

---

## 19.8 Build Process

### Maven Lifecycle

Clean

↓

Validate

↓

Compile

↓

Test

↓

Package

↓

Verify

↓

Install

Artifacts

- Spring Boot JAR
- Docker Image
- Test Reports
- Coverage Reports

---

## 19.9 Code Quality Gates

Before deployment:

- Build Success
  - Unit Tests Passed
  - Integration Tests Passed
  - Code Coverage  $\geq 85\%$
  - No Critical Security Issues
  - No Blocker Bugs
  - SonarQube Quality Gate Passed
-

# 19.10 Docker Build

## Artifacts

Backend Image

Frontend Image

Nginx Image

## Image Naming

smarteducation-backend:1.0.0

smarteducation-frontend:1.0.0

---

# 19.11 Docker Registry

Images stored in:

- Docker Hub
  - GitHub Container Registry
  - Private Registry (Recommended)
- 

# 19.12 Deployment Environments

Development

↓

QA

↓

UAT

↓

Production

Each environment has:

- Separate Database
- Separate Redis

- Separate Storage
- Independent Configuration

---

## 19.13 Environment Configuration

### Profiles

application-dev.yml

application-qa.yml

application-uat.yml

application-prod.yml

Configuration includes:

- Database
- Redis
- AI API Keys
- SMTP
- MinIO
- Logging

Sensitive values are stored securely using environment variables or secret management systems.

---

## 19.14 Deployment Workflow

Build Success

↓

Docker Image

↓

Deploy to QA

↓

QA Testing

↓

Deploy to UAT

↓

Business Approval

↓

Deploy to Production

---

## 19.15 Release Strategy

### Release Types

- Major Release
- Minor Release
- Patch Release
- Hotfix Release

### Example

v1.0.0

v1.1.0

v1.1.1

---

## 19.16 Deployment Strategies

### Supported Strategies

#### Rolling Deployment

- Gradual replacement
  - Minimal downtime
- 

#### Blue-Green Deployment

Blue Environment

↓

Switch Traffic

↓

Green Environment

### Benefits

- Easy rollback
- Near-zero downtime

---

## 19.17 Rollback Strategy

### Rollback triggers

- Deployment Failure
- High Error Rate
- Critical Bug
- Health Check Failure

### Workflow

Deployment

↓

Failure

↓

Previous Version

↓

Rollback

↓

Health Check

↓

Service Restored

---

## 19.18 Database Migration

### Tool

- Flyway

## Migration Flow

Application Start

↓

Flyway Migration

↓

Database Updated

↓

Application Ready

All schema changes must be version-controlled.

---

## 19.19 Infrastructure Automation

### Automated Tasks

- Build
- Test
- Deployment
- Docker Image Creation
- Database Migration
- Health Check
- Monitoring Registration

### Future

- Terraform
  - Ansible
- 

## 19.20 Production Health Checks

### Endpoints

/actuator/health

/actuator/info

/actuator/prometheus

Deployment succeeds only after successful health checks.

---

## 19.21 Secrets Management

Secrets include:

- JWT Secret
- Database Password
- SMTP Credentials
- AI API Keys
- Redis Password
- MinIO Credentials

Secrets must never be committed to source control.

---

## 19.22 Monitoring After Deployment

Monitor

- CPU
  - Memory
  - API Response Time
  - Error Rate
  - Redis
  - Database
  - AI Services
  - Storage
  - Active Users
- 

## 19.23 Release Checklist

Before Production Release

- Feature Complete
- Code Review Approved
- Tests Passed
- Security Scan Passed



- Database Migration Verified
  - Docker Image Built
  - Backup Completed
  - Release Notes Prepared
  - Rollback Plan Ready
- 

## 19.24 Package Structure

devops

```
|— docker
|— github-actions
|— nginx
|— scripts
|— monitoring
|— deployment
└— backup
```

---

## 19.25 CI/CD Monitoring

Track

- Build Time
  - Deployment Time
  - Success Rate
  - Failed Builds
  - Rollbacks
  - Test Coverage
  - Pipeline Duration
- 

## 19.26 Testing in Pipeline

Automatically execute

- Unit Tests

- Integration Tests
- API Tests
- Security Tests
- Static Code Analysis
- Code Coverage

Deployment stops if mandatory quality gates fail.

---

## 19.27 Best Practices

- Keep pipelines fast and repeatable.
  - Automate testing at every stage.
  - Use immutable Docker images.
  - Version all releases.
  - Store secrets securely.
  - Keep deployments reproducible.
  - Use infrastructure as code where possible.
  - Maintain rollback capability.
  - Monitor every deployment.
- 

## 19.28 Acceptance Criteria

The CI/CD & DevOps framework shall be accepted when:

- Builds are triggered automatically on code changes.
  - Automated tests execute successfully.
  - Docker images are generated and versioned.
  - Deployments progress through Dev, QA, UAT, and Production environments.
  - Health checks validate successful deployments.
  - Rollback procedures restore previous versions when required.
  - Secrets remain protected.
  - Monitoring confirms system stability after deployment.
- 

## 19.29 Complete DevOps Workflow

Developer

↓

Git Commit

↓

GitHub

↓

CI Pipeline

↓

Build

↓

Testing

↓

Code Quality

↓

Docker Build

↓

Deploy QA

↓

Deploy UAT

↓

Production Approval

↓

Production Deployment

↓

Monitoring

↓

Feedback

---

## 19.30 Chapter Summary

The **CI/CD Pipeline, DevOps Workflow & Release Management** architecture establishes an automated, secure, and reliable software delivery process for the Smart Education Platform. Through Git-based workflows, automated testing, containerized deployments, environment-specific configurations, quality gates, health checks, rollback mechanisms, and continuous monitoring, the platform achieves faster release cycles, improved software quality, and operational stability.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 20 – Performance Tuning, Scalability, High Availability & Production Readiness

Version: 1.0

---

### 20.1 Introduction

The **Performance Tuning, Scalability, High Availability (HA), and Production Readiness** architecture ensures that the Smart Education Platform delivers fast response times, remains available under heavy load, and scales efficiently as the number of learners, recruiters, and administrators grows.

This chapter defines optimization strategies across the application, database, infrastructure, network, caching, storage, and deployment layers.

The architecture is designed to support **100,000+ learners** in Phase 1 and is extensible to **millions of users** in future releases.

---

### 20.2 Objectives

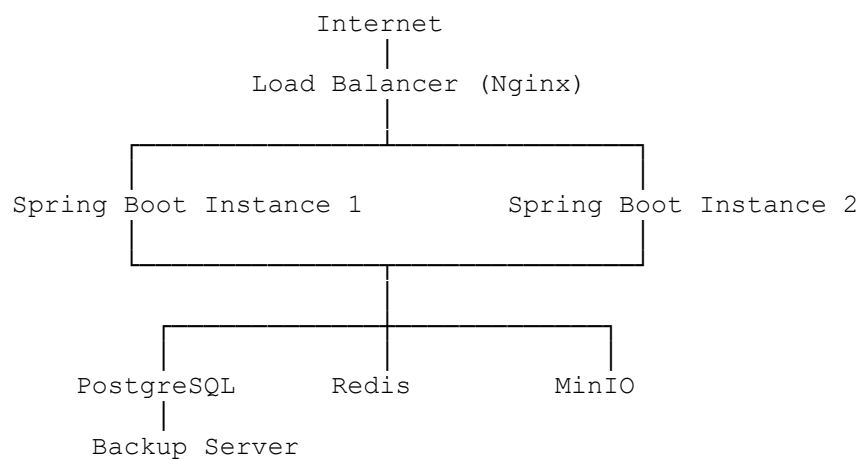
The Production Architecture shall:

- Deliver low response times

- Handle high concurrent traffic
- Eliminate single points of failure
- Support horizontal scaling
- Ensure high availability
- Minimize downtime
- Provide disaster recovery capabilities
- Maintain production stability

---

## 20.3 Production Architecture



---

## 20.4 Performance Targets

| Component         | Target      |
|-------------------|-------------|
| Login API         | < 500 ms    |
| Dashboard         | < 2 Seconds |
| Coding Submission | < 5 Seconds |
| SQL Submission    | < 3 Seconds |
| Resume Upload     | < 3 Seconds |
| Job Search        | < 1 Second  |
| AI Coach Response | < 5 Seconds |
| Cache Response    | < 50 ms     |

---

## 20.5 Application Performance Optimization

## Strategies

- Constructor Dependency Injection
  - Lazy Loading
  - Efficient DTO Mapping
  - Asynchronous Processing
  - Pagination
  - Batch Processing
  - Connection Pooling
  - Redis Caching
  - Optimized Serialization
- 

# 20.6 Database Performance Tuning

## Optimization Techniques

- Proper Indexing
- Composite Indexes
- Query Optimization
- Prepared Statements
- Batch Inserts
- Read-only Transactions
- Partitioning (Future)
- Connection Pool Optimization

Indexes should be created for:

- Email
  - Mobile Number
  - Job Status
  - Placement Score
  - Leaderboard Rank
  - Created Date
- 

# 20.7 Hibernate Optimization

## Best Practices

- Lazy Fetching
- EntityGraph

- DTO Projection
- Batch Fetch Size
- Optimistic Locking
- Avoid N+1 Queries

### Recommended Settings

```
hibernate.jdbc.batch_size=50
```

```
hibernate.order_inserts=true
```

```
hibernate.order_updates=true
```

```
hibernate.generate_statistics=false
```

---

## 20.8 JVM Performance Tuning

### Recommended JVM Settings

```
-Xms2G
```

```
-Xmx4G
```

```
-XX:+UseG1GC
```

```
-XX:MaxGCPauseMillis=200
```

```
-XX:+HeapDumpOnOutOfMemoryError
```

### Benefits

- Reduced GC pauses
  - Stable memory usage
  - Better throughput
- 

## 20.9 Redis Optimization

Redis is used for:

- Dashboard Cache
- Leaderboards
- OTP
- Configuration
- AI Cache

- Job Listings

#### Best Practices

- TTL for every cache
  - Cache invalidation after updates
  - Avoid oversized objects
  - Monitor memory usage
  - Use compression where beneficial
- 

## 20.10 API Performance

#### Recommendations

- Response Compression (GZIP)
  - Pagination
  - DTO Projection
  - Async Processing
  - HTTP Keep-Alive
  - Efficient JSON Serialization
- 

## 20.11 File Upload Optimization

#### Techniques

- Multipart Upload
  - Streaming Upload
  - Image Compression
  - Thumbnail Generation
  - Background Processing
  - CDN Support (Future)
- 

## 20.12 Asynchronous Processing

#### Run asynchronously

- Email



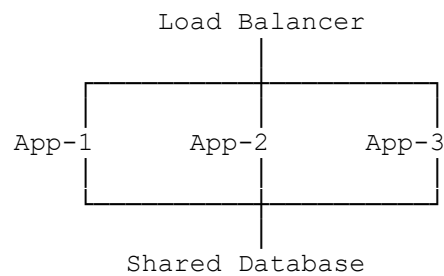
- SMS
- Notifications
- AI Requests
- Report Generation
- PDF Generation

Implementation

@Async

---

## 20.13 Horizontal Scaling



Benefits

- Increased capacity
  - Better fault tolerance
  - Reduced response time
- 

## 20.14 Vertical Scaling

Upgrade

- CPU
- RAM
- SSD Storage
- Network Bandwidth

Suitable for early growth before horizontal expansion.

---

## 20.15 Load Balancing

Recommended

- Nginx
- HAProxy (Future)

Load Balancing Methods

- Round Robin
- Least Connections
- IP Hash

---

## 20.16 High Availability (HA)

High Availability Components

- Multiple Application Servers
- Database Backup
- Redis Persistence
- MinIO Replication
- Health Monitoring
- Automatic Failover (Future)

Target Availability

**99.9% Uptime**

---

## 20.17 Database High Availability

Future Architecture

Primary PostgreSQL

↓

Streaming Replication

↓

Read Replica

↓

Automatic Failover

## Benefits

- Read scalability
  - Disaster recovery
  - Reduced downtime
- 

# 20.18 Redis High Availability

## Future Support

- Redis Sentinel
- Redis Cluster

## Provides

- Automatic failover
  - Data replication
  - High availability
- 

# 20.19 Disaster Recovery (DR)

## Recovery Plan

- Daily Database Backup
- Weekly Full Backup
- MinIO Backup
- Configuration Backup
- Restore Validation
- Disaster Recovery Drills

## Recovery Targets

- **RPO (Recovery Point Objective):**  $\leq 15$  minutes
  - **RTO (Recovery Time Objective):**  $\leq 1$  hour
- 

# 20.20 Capacity Planning

### Initial Capacity

- 100,000 Learners
- 5,000 Recruiters
- 50 Super Admins

### Expected Growth

- 1 Million+ Users (Future)
- 

## 20.21 Monitoring

### Monitor

- CPU Usage
  - Memory Usage
  - JVM Heap
  - Database Connections
  - Redis Memory
  - Disk Space
  - API Response Time
  - Active Sessions
  - Queue Size
  - Error Rate
- 

## 20.22 Production Security

### Security Measures

- HTTPS
  - WAF (Future)
  - JWT Authentication
  - Rate Limiting
  - Secure Headers
  - Database Encryption
  - Secret Management
  - Firewall Rules
-

## 20.23 Production Checklist

Before Go-Live

- All Tests Passed
  - Code Review Completed
  - Security Scan Completed
  - Database Migration Completed
  - Backup Verified
  - Monitoring Enabled
  - Logging Enabled
  - SSL Configured
  - Health Checks Passed
  - Rollback Plan Ready
- 

## 20.24 SLA & SLO

### Service Level Agreement (SLA)

- Availability: **99.9%**
  - Planned Maintenance Window: Announced in advance
  - Critical Issue Response:  $\leq 30$  minutes
- 

### Service Level Objectives (SLO)

- API Success Rate  $\geq 99.5\%$
  - Login Success  $\geq 99.9\%$
  - Dashboard Load  $\leq 2$  Seconds
  - Error Rate  $< 1\%$
- 

## 20.25 Production Readiness Checklist

Application

- Spring Boot Optimized
- JVM Tuned

- Health Checks Enabled

#### Database

- Indexes Created
- Backup Configured
- Performance Verified

#### Infrastructure

- Docker Ready
- Reverse Proxy Configured
- Monitoring Enabled

#### Security

- JWT Enabled
- HTTPS Enabled
- Secrets Protected

#### Operations

- Logs Verified
- Alerts Configured
- Runbooks Available

---

## 20.26 Future Scalability

#### Phase 2

- Kubernetes
- Auto Scaling
- Service Mesh
- Kafka
- Microservices
- CDN
- Multi-Region Deployment
- Read Replicas

---

## 20.27 Performance Testing

## Validate

- 100 Concurrent Users
- 500 Concurrent Users
- 1,000 Concurrent Users
- Stress Testing
- Spike Testing
- Endurance Testing

## Acceptance Targets

- No data loss
  - Stable response times
  - Graceful degradation under heavy load
- 

# 20.28 Best Practices

- Cache frequently accessed data.
  - Optimize database queries.
  - Keep APIs stateless.
  - Use asynchronous processing for long-running tasks.
  - Monitor system health continuously.
  - Implement graceful error handling.
  - Perform regular backups.
  - Test disaster recovery procedures.
  - Review capacity regularly.
  - Plan infrastructure growth proactively.
- 

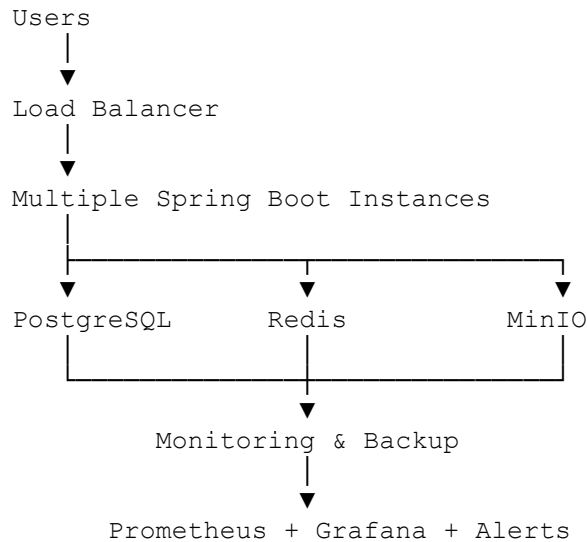
# 20.29 Acceptance Criteria

The platform shall be considered production-ready when:

- Performance targets are achieved.
- Horizontal scaling works successfully.
- High availability architecture is operational.
- Disaster recovery procedures are documented and tested.
- Monitoring and alerting are active.
- Production security controls are enabled.
- Load and stress tests pass successfully.
- SLA and SLO objectives are met.

---

## 20.30 Production Readiness Architecture



---

## 20.31 Chapter Summary

The **Performance Tuning, Scalability, High Availability & Production Readiness** architecture equips the Smart Education Platform for enterprise-scale deployment. By combining application optimization, database tuning, intelligent caching, asynchronous processing, horizontal scalability, high availability, disaster recovery planning, and continuous monitoring, the platform delivers a resilient, secure, and high-performance environment capable of supporting large-scale educational ecosystems with confidence.

## Low-Level Design (LLD)

### Smart Education Platform (Phase 1)

## Chapter 21 – Low-Level Design Summary, Development Guidelines & Implementation Roadmap

Version: 1.0



---

## 21.1 Introduction

This chapter summarizes the complete **Low-Level Design (LLD)** of the Smart Education Platform (Phase 1) and provides a practical roadmap for implementation. It defines the recommended development sequence, coding guidelines, sprint planning, team responsibilities, project governance, quality standards, and deployment readiness.

The objective is to ensure that every team member follows a common implementation approach, resulting in a scalable, maintainable, secure, and production-ready application.

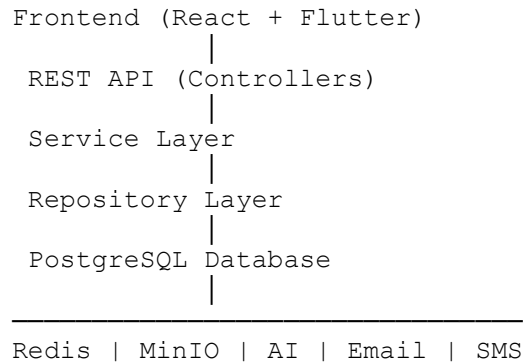
---

## 21.2 LLD Coverage Summary

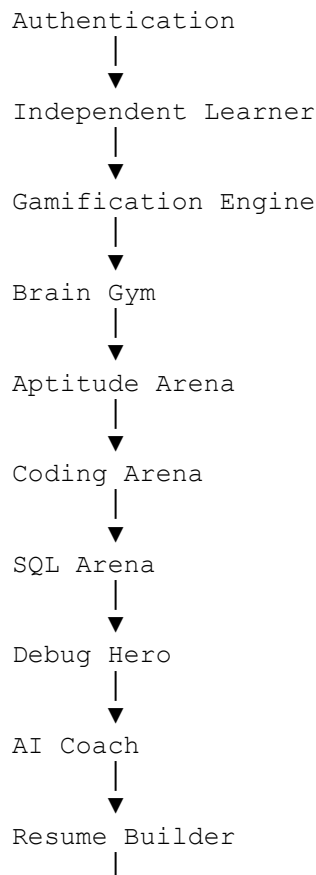
The Low-Level Design includes the following chapters:

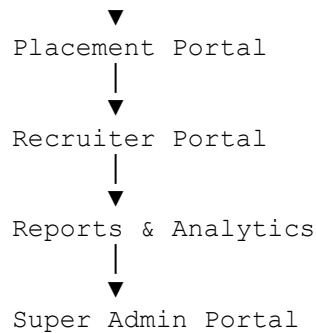
| Chapter | Module                                     |
|---------|--------------------------------------------|
| 1       | LLD Introduction & Architecture Principles |
| 2       | Package Structure & Module Design          |
| 3       | Database Entity Design & JPA Architecture  |
| 4       | DTO & Object Mapping                       |
| 5       | Repository Layer                           |
| 6       | Service Layer                              |
| 7       | Controller Layer                           |
| 8       | Spring Security & JWT                      |
| 9       | Validation & Exception Handling            |
| 10      | Redis Caching & Session Management         |
| 11      | File Storage & Document Management         |
| 12      | AI Integration Architecture                |
| 13      | Notification System                        |
| 14      | Scheduler & Background Jobs                |
| 15      | Logging & Monitoring                       |
| 16      | UML & Module Integration                   |
| 17      | Design Patterns & Coding Standards         |
| 18      | Testing Framework                          |
| 19      | CI/CD & DevOps                             |
| 20      | Performance & Production Readiness         |

## 21.3 Complete Development Architecture



## 21.4 Module Dependency Matrix





## 21.5 Recommended Development Sequence

### Phase 1 – Foundation

- Spring Boot Project Setup
  - PostgreSQL Configuration
  - JPA Configuration
  - Flyway Migration
  - Global Configuration
  - Common Utilities
- 

### Phase 2 – Authentication

- User Registration
  - Login
  - JWT
  - Refresh Token
  - Role Management
- 

### Phase 3 – Core Platform

- Independent Learner Dashboard
  - Profile Management
  - Gamification Engine
  - Notifications
-

## **Phase 4 – Assessment Modules**

- Brain Gym
  - Aptitude Arena
  - Coding Arena
  - SQL Arena
  - Debug Hero
- 

## **Phase 5 – AI Features**

- AI Coach
  - Resume AI Review
  - Mock Interview AI
- 

## **Phase 6 – Career Features**

- Resume Builder
  - Placement Portal
  - Recruiter Portal
- 

## **Phase 7 – Administration**

- Reports
  - Super Admin
  - Configuration
  - Audit
- 

## **Phase 8 – Production**

- Testing
  - Performance Tuning
  - CI/CD
  - Monitoring
  - Deployment
-

## 21.6 Sprint-wise Roadmap

| <b>Sprint</b> | <b>Deliverables</b>           |
|---------------|-------------------------------|
| Sprint 1      | Project Setup, Authentication |
| Sprint 2      | Learner Dashboard, Profile    |
| Sprint 3      | Gamification Engine           |
| Sprint 4      | Brain Gym & Aptitude          |
| Sprint 5      | Coding Arena                  |
| Sprint 6      | SQL Arena & Debug Hero        |
| Sprint 7      | AI Coach                      |
| Sprint 8      | Resume Builder                |
| Sprint 9      | Placement Portal              |
| Sprint 10     | Recruiter Portal              |
| Sprint 11     | Reports & Analytics           |
| Sprint 12     | Super Admin Portal            |
| Sprint 13     | Testing & Bug Fixing          |
| Sprint 14     | Production Deployment         |

---

## 21.7 Team Responsibilities

### Backend Team

Responsibilities

- Spring Boot
  - APIs
  - Database
  - Security
  - AI Integration
  - Redis
  - MinIO
- 

### Frontend Team

Responsibilities

- React Web

- Flutter Mobile
  - API Integration
  - Responsive UI
  - State Management
- 

## **UI/UX Team**

### Responsibilities

- Design System
  - Wireframes
  - Figma
  - Components
  - User Experience
- 

## **QA Team**

### Responsibilities

- Functional Testing
  - API Testing
  - Regression Testing
  - Automation Testing
  - UAT Support
- 

## **DevOps Team**

### Responsibilities

- CI/CD
  - Docker
  - Deployment
  - Monitoring
  - Backup
  - Security
-

## 21.8 Coding Guidelines

All developers shall:

- Follow SOLID Principles
  - Use Constructor Injection
  - Write Clean Code
  - Follow Naming Standards
  - Use DTOs
  - Write Unit Tests
  - Document Public APIs
  - Avoid Code Duplication
- 

## 21.9 Git Workflow

Feature Branch

↓

Pull Request

↓

Code Review

↓

Develop Branch

↓

QA

↓

Production

---

## 21.10 Definition of Done (DoD)

A feature is complete only when:

- Requirements implemented
- Code reviewed
- Unit tests passed

- Integration tests passed
  - API documented
  - Security validated
  - Performance verified
  - QA approved
  - Product Owner approved
  - Merged into main branch
- 

## 21.11 Documentation Checklist

Each module must include:

- PRD
  - SRS
  - LLD
  - API Documentation
  - Database Scripts
  - Test Cases
  - Deployment Guide
  - User Manual
  - Release Notes
- 

## 21.12 Quality Checklist

Before release:

- No blocker bugs
  - No critical vulnerabilities
  - Code coverage  $\geq 85\%$
  - SonarQube Quality Gate passed
  - Performance benchmarks achieved
  - Documentation complete
- 

## 21.13 Risk Assessment

| Risk                | Mitigation       |
|---------------------|------------------|
| AI Service Downtime | Retry & Fallback |



| <b>Risk</b>        | <b>Mitigation</b>                |
|--------------------|----------------------------------|
| Database Failure   | Backup & Replication             |
| High Traffic       | Redis + Load Balancer            |
| Security Attack    | JWT, Rate Limiting, HTTPS        |
| Deployment Failure | Automated Rollback               |
| Data Loss          | Daily Backup & Disaster Recovery |

---

## 21.14 Implementation Timeline

Month 1  
 Project Setup  
 Authentication  
 Core Architecture

↓

Month 2  
 Learner Dashboard  
 Gamification  
 Brain Gym  
 Aptitude

↓

Month 3  
 Coding  
 SQL  
 Debug Hero

↓

Month 4  
 AI Coach  
 Resume Builder  
 Placement

↓

Month 5  
 Recruiter Portal  
 Reports  
 Admin Portal

↓

Month 6  
 Testing  
 Optimization  
 Deployment  
 Go Live

---

## 21.15 Production Readiness Checklist

Before launch ensure:

- Authentication secured
- All APIs tested
- Database optimized
- Redis configured
- Monitoring enabled
- Logging enabled
- SSL configured
- Backup configured
- Alerts configured
- CI/CD operational

---

## 21.16 Future Roadmap (Phase 2)

Planned enhancements:

- Learning Module
- Video Learning Platform
- Live Classes
- Mentor Portal
- Parent Portal
- AI Learning Assistant
- Mobile Offline Mode
- Microservices Architecture
- Kubernetes Deployment
- Real-Time Collaboration

---

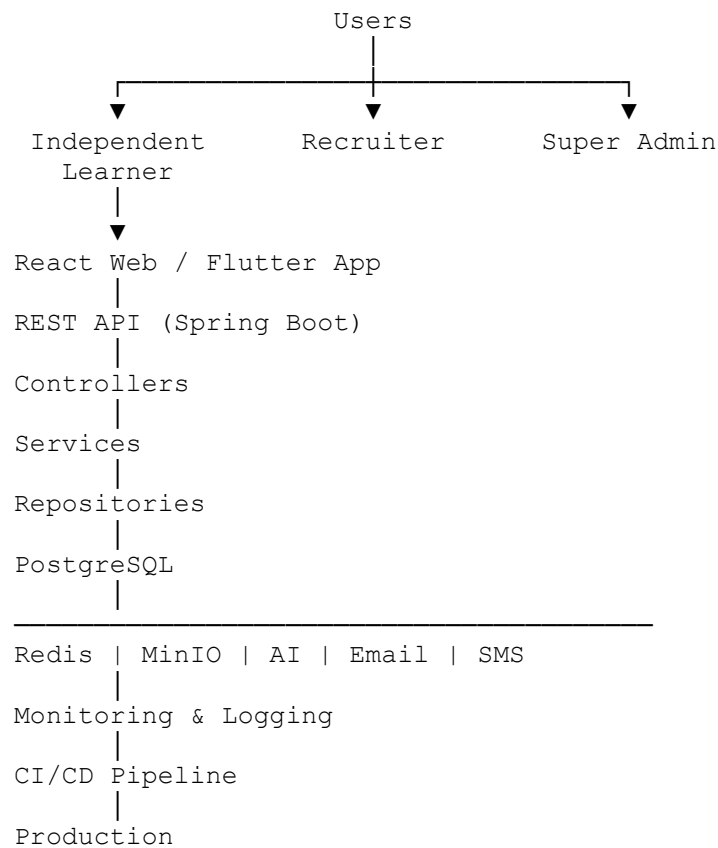
## 21.17 Key Success Factors

The project will succeed by focusing on:

- Modular architecture
- Secure development
- Automated testing

- Performance optimization
  - Continuous monitoring
  - User-centric design
  - Scalable infrastructure
  - Comprehensive documentation
- 

## 21.18 Final Architecture Overview



## 21.19 Low-Level Design Deliverables

The completed LLD provides:

- Complete Package Structure
- JPA Entity Design
- DTO Architecture
- Repository Design
- Service Layer Design

- Controller Design
  - Security Architecture
  - Validation Framework
  - Redis Caching Strategy
  - File Storage Architecture
  - AI Integration Design
  - Notification Framework
  - Scheduler & Automation
  - Logging & Monitoring
  - UML Diagrams
  - Design Patterns
  - Testing Framework
  - DevOps & CI/CD
  - Performance & Scalability
  - Implementation Roadmap
- 

## 21.20 Final Conclusion

The **Low-Level Design (LLD)** serves as the complete engineering blueprint for the **Smart Education Platform (Phase 1)**. Together with the **Product Requirements Document (PRD)** and **Software Requirements Specification (SRS)**, it provides end-to-end guidance for design, development, testing, deployment, and maintenance.

This architecture delivers:

- **Three user portals:** Independent Learner, Recruiter, and Super Admin
- **Assessment modules:** Brain Gym, Aptitude Arena, Coding Arena, SQL Arena, and Debug Hero
- **Gamification Engine:** XP, Levels, Coins, Badges, Missions, Rewards, and Leaderboards
- **AI-powered features:** AI Coach, Resume Review, Career Guidance, and Mock Interviews
- **Career tools:** Resume Builder and Placement Portal
- **Enterprise-grade architecture:** Spring Boot, React, Flutter, PostgreSQL, Redis, MinIO, JWT Security, Docker, and CI/CD
- **Production readiness:** Performance optimization, monitoring, security, testing, high availability, and DevOps automation

This concludes the **Low-Level Design (LLD)** document for **Smart Education Platform – Phase 1**.